EP-392: UG Project-2

Downscaling Approach To Produce High Spatial Soil Moisture Products Using Satellite Data

Mayur Kunnitigi Matada - 23174017

Supervisor: Prof. Rajendra Prasad

Department Of Physics

October 2026



Contents

From Regional Foundations to Global Scale **6**

Abstract **7**

Executive Summary **8**

1 Project Motivation, Scope, and Scientific Question **10**

1.1 Soil moisture as a physical state variable 10

1.2 Why downscaling and fusion are needed 10

1.3 The central research question 11

1.4 Scope and non-claims 11

2 Architecture of the Entire Scientific-Computing System **12**

2.1 System-level view 12

2.2 Scale of the project 14

3 Global Data Contract and Ground Truth **15**

3.1 Why a contract is necessary 15

3.2 Depth and quality 15

3.3 Archive audit and streaming conversion 15

3.4 Ground-truth statistics 16

4 Data Sources, Provenance, and Environmental Predictors **17**

4.1 Data sources and providers 17

4.2 Provenance map 19

4.3 Copernicus ERA5-Land 19

4.4 NASA SMAP . . . 19
4.5 MODIS from Planetary Computer . . . 19
4.6 Sentinel-1 RTC from Planetary Computer . . . 20
4.7 Static layers from Google Earth Engine . . . 20

5 Remote Acquisition and Distributed Data Engineering 21
5.1 Why remote data extraction became the dominant systems problem . . . 21
5.2 Cell reduction and collision maps . . . 21
5.3 Scene clustering . . . 22
5.4 Azure Poland processing client . . . 22
5.5 Resume engineering . . . 23
5.6 Overview versus exact reads . . . 23

6 Storage Formats, Merge, and Alignment 24
6.1 Why no single file format is sufficient . . . 24
6.2 Low-memory merge . . . 25
6.3 The row-order hazard . . . 25

7 The Optimization Programme 26
7.1 The counterfactual . . . 26
7.2 Optimization principle one: do not create data that will be discarded . . . 26
7.3 Optimization principle two: amortize remote fixed costs . . . 27
7.4 Optimization principle three: cache at the storage grain . . . 27
7.5 Optimization principle four: move deterministic work off the GPU . . . 27
7.6 Optimization principle five: bound parallelism . . . 28
7.7 Optimization principle six: preserve the effective batch under finite VRAM 28
7.8 Optimization principle seven: make caches self-describing . . . 28
7.9 Optimization register in prose . . . 28
7.10 What the optimisation means in computer terms . . . 29
7.11 Why the unoptimised experiment was not practical . . . 30
7.11.1 A simple analogy for the processing pipeline . . . 30
7.11.2 Network traffic: reduce requests before reducing workers . . . 31

7.11.3 Why overview reads helped, and what they changed 31
7.11.4 Memory: why a correct calculation can still crash 31
7.11.5 GPU memory: microbatches preserve the learning problem 32
7.11.6 CPU GPU balance: keeping the expensive machine occupied . . . 32
7.11.7 Storage formats: each format removes a different bottleneck . . . 33
7.11.8 Restartability: saving time when failure is expected 33
7.11.9 The resulting saving in practical terms 34

8 Engineering Internals of the Global Run 35
8.1 The unit of computation was redesigned several times 35
8.2 Storage layers and why no single format was sufficient 36
8.3 Resumable extraction as a state machine 38
8.4 Request geometry, compression, and network cost 39
8.5 Cache signatures and invalidation 39
8.6 Memory traffic during training 39

9 Model Architecture and Physical Interpretation 41
9.1 Final v19.5 configuration 41
9.2 Full model diagram 42
9.3 Spatial encoders 42
9.4 Temporal TCN 42
9.5 Static and location branches 43
9.6 Modality gate and fusion 43
9.7 Calibration 43

10 Training, Numerical Stability, and Reproducibility 44
10.1 Hyperparameter search 44
10.2 Loss and optimisation 44
10.3 AMP, accumulation, and EMA 45
10.4 Compilation and hardware dependence 45
10.5 Checkpoint policy 45

11 Evaluation and Error Analysis **46**

11.1 Evaluation philosophy 46

11.2 Primary results 46

11.3 Temporal and regional diagnostics 47

11.4 Figures from the global analysis 47

11.5 Oracle trimming 48

12 Extended Diagnostic Gallery and Technical Case Studies **50**

12.1 Why the gallery is part of the result 50

12.2 Monthly and seasonal behaviour 50

12.3 Moisture range and residual structure 51

12.4 Raw versus calibrated output 53

12.5 Paired station and regional comparisons 55

12.6 Worst-station and event-context analyses 56

12.7 What the gallery establishes 57

13 Technical Hurdles and Solutions **58**

13.1 Data association was harder than tensor arithmetic 58

13.2 Same date is not necessarily same physical state 58

13.3 Calibration can dominate the headline score 58

13.4 Location is useful and dangerous 59

13.5 Missingness can become a hidden predictor 59

13.6 Physical validity exceeds numerical validity 59

14 Reproducibility, Audit Trail, and Release **60**

14.1 What is recorded 60

14.2 Failure should be explicit 61

14.3 Reproduction sequence 61

14.4 Change control across providers 62

14.5 Why auditability matters scientifically 63

15 Conclusion **64**

16 Future Work **65**

16.1 Certification of physical transfer 65

16.2 Depth-specific and temporally synchronous targets 65

16.3 Uncertainty and statistically dependent observations 65

16.4 Physical stress tests 66

16.5 Representation and approximation ablations 66

16.6 Locked certification release 66

16.7 Operational data system 66

From Regional Foundations to Global Scale

Stage One established the regional foundation for this project through a focused study of soil-moisture downscaling over Spain. The work used REMEDHUS station observations as the reference target and combined them with satellite, meteorological, soil, and terrain predictors. It first developed a quality controlled data pipeline and a residual XGBoost baseline, and then used the same carefully checked inputs to build a multimodal spatial-temporal deep model. Evaluation was performed under an unseen-station protocol so that the reported improvement measured transfer to stations not used for training, rather than simply memorisation of randomly held-out rows.

The main outcome of Stage One was therefore more than a single accuracy value. It established the data-cleaning rules, daily target construction, patch-based feature representation, residual anchoring strategy, and station-disjoint evaluation approach that support the wider project. It also showed why neighbourhood context, temporal information, and explicit handling of the coarse soil-moisture anchor are useful when a point observation is compared with area-integrated satellite and meteorological measurements.

Stage One identified globalization as the next major step. A regional result over Spain cannot by itself demonstrate that the method transfers across climate zones, soil types, vegetation regimes, station networks, and satellite coverage conditions. Stage Two addresses that future-work objective by extending the system to a global, multi-provider dataset and by redesigning the acquisition, storage, alignment, caching, training, and audit machinery for millions of station-date examples. The global system retains the scientific lessons of Stage One while making the computational assumptions explicit enough to support a much larger and more diverse experiment.

Abstract

This report documents the construction of a global, multi-modal soil-moisture learning system from raw observation archives through remote data acquisition, storage engineering, model training, calibration, and scientific error analysis. The project is substantially larger than a conventional machine-learning experiment: its reported implementation footprint is 143,850 lines of code, including archive parsers, scientific data contracts, cloud-raster clients, distributed-work recovery, array conversion, validation, model training, and diagnostic analysis.

The target is near-surface volumetric soil moisture in m^3/m^3. The reference observations come from the International Soil Moisture Network (ISMN) and are quality filtered, depth classified, and aggregated to UTC calendar-day targets. Predictors combine Copernicus ERA5-Land meteorological forcing, NASA SMAP microwave context, MODIS and Sentinel-1 products hosted by Microsoft Planetary Computer, and static soil and terrain layers extracted through Google Earth Engine. The provenance distinction is essential: Google Earth Engine supplies static layers only; Planetary Computer remains the authoritative host for the dynamic raster products; and a paid Azure Poland virtual machine was used as the high-throughput processing client after network optimisation.

The final v19.5 model uses modality-specific residual spatial encoders, global mean/-max/attention pooling, a temporal convolutional network with irregular history masking and last/history gating, static and location branches, a learned five-way modality gate, nonlinear fusion, and an identity-initialised quadratic calibration layer. The location-history configuration reaches calibrated test RMSE 0.06594 and $R^2 = 0.65341$ over 905,248 rows; the environment-only configuration reaches RMSE 0.08626 and $R^2 = 0.47046$ over 644,716 rows.

The report emphasizes technical feats and their scientific boundaries. The system reduces a naive estimate of roughly 700 GB of data and 60 days of top-tier GPU computation through scene clustering, COG overview reads, resumable shards, chunk-level caching, memory-mapped arrays, train-only FP16 normalisation, bounded prefetch, gradient accumulation, and staged hyperparameter search. These are not merely performance tricks: they determine whether the global experiment can be repeated, audited, and interpreted.

Executive Summary

Project Overview

The project transformed heterogeneous, irregular, multi-provider global observations into a verified and resumable scientific-computing pipeline that can train and evaluate a multi-modal neural model at a scale that would be impractical under naive data and compute assumptions.

Quantified scale

- 143,850 reported lines of implementation.
- 4,340 ISMN station files across 61 networks.
- 144,466,562 observations audited.
- 103,504,765 good-quality observations retained in the audit.
- 5,045,849 daily Parquet rows and 3,514,643 unique station-date samples.
- 204,033 unique Sentinel-1 scenes indexed.
- 528,280 scene-cluster associations recorded.
- 21,655 resumable NPZ scene shards produced.
- 905,248 held-out location-history predictions evaluated.

What makes the work technically difficult

The data sources do not share a time grid, spatial grid, resolution, missingness pattern, quality convention, storage format, or network access mechanism. A station observation is a point-like measurement at a particular depth; a satellite product is an area-integrated

measurement whose acquisition date may be offset from the target; reanalysis is a gridded forcing field; static layers describe slow site properties; and a microwave product such as SMAP has a large footprint. The model is therefore only one stage of the scientific problem.

The project solved data identity before model identity. A feature array with the right shape can still be scientifically wrong if its row order has drifted. Stable sample identifiers, producer row indexes, sidecar tables, completion flags, shape checks, sampled reads, and checksums were used to make association auditable.

The project solved remote request geometry before simply adding more workers. STAC searches and COG reads were clustered by shared scene and time window, overview reads were benchmarked against the 1 km target, and finished scene units were persisted so failures did not force a global restart.

The project solved model throughput before treating GPU size as the only limitation. Normalised FP16 memory maps, vectorised gathers, bounded queues, pinned memory, asynchronous copies, compilation fallback, microbatches, gradient accumulation, and EMA reduced the wall-clock cost and the failure rate of training.

Summarized Results

Configuration	Test rows	RMSE	MAE	R^2	Robust R^2
Environment-only	644,716	0.08626	0.06794	0.47046	0.51276
Location-history	905,248	**0.06594**	**0.04854**	**0.65341**	**0.74078**

The location-history result is a calibrated result. Its raw test RMSE is approximately 0.11190 before the learned output calibration. The report therefore distinguishes raw, calibrated, robust, and anchor comparisons rather than presenting one score as if it represented every layer of the system.

Chapter 1

Project Motivation, Scope, and Scientific Question

1.1 Soil moisture as a physical state variable

Soil moisture is the water held in soil pore space and is commonly represented as volumetric water content. In the surface layer it responds to precipitation, infiltration, evaporation, transpiration, runoff, soil texture, terrain, vegetation, and atmospheric demand. It is not a passive scalar that changes uniformly across a landscape. Two neighbouring locations can respond differently to the same rainfall because their storage and drainage properties differ.

The physical importance is broad. Surface moisture influences land-atmosphere energy exchange, plant stress, drought evolution, runoff generation, and irrigation demand. The project is therefore motivated by a measurement problem: in-situ stations provide high-value local observations but are sparse, whereas remotely sensed products provide spatial coverage but are indirect, coarse, intermittent, or affected by clouds, vegetation, geometry, and footprint mismatch.

1.2 Why downscaling and fusion are needed

A coarse microwave retrieval can provide a useful large-scale state estimate while averaging over fields, roads, terrain, and vegetation communities. A finer-scale product must use auxiliary information to infer local variation. This is not the same as creating new measurements. It is a conditional reconstruction constrained by available observations and learned relationships.

The global system combines:

1. An in-situ target with explicit quality and depth policy;
2. Optical vegetation and thermal information from MODIS;
3. Radar backscatter structure from Sentinel-1;
4. Meteorological forcing and memory from ERA5-Land;
5. A coarse microwave anchor from SMAP;
6. Static texture and terrain from GEE exports;
7. Location and station history in one controlled model configuration.

1.3 The central research question

The operational research question is:

> Can a verified multi-modal system estimate daily near-surface soil moisture at 1 km station-centred support from heterogeneous global observations while retaining enough provenance, computational efficiency, and diagnostic transparency for scientific interpretation?

This question contains three subquestions:

1. Can the data sources be harmonised without silently changing the physical target?
2. Can the remote-sensing workload be executed at global scale without repeated or unrecoverable work?
3. Does a model with spatial, temporal, static, location, and anchor pathways outperform simpler environmental-only alternatives under station/network-aware evaluation?

1.4 Scope and non-claims

The model estimates the observed target distribution; it does not solve a complete soil-water balance, perform causal inference, or guarantee physical conservation. The report does not claim that a high R^2 proves universal geographic transfer. It also does not treat oracle trimming as a deployment filter or confuse the Azure processing client with the Planetary Computer archive.

Chapter 2

Architecture of the Entire Scientific-Computing System

2.1 System-level view

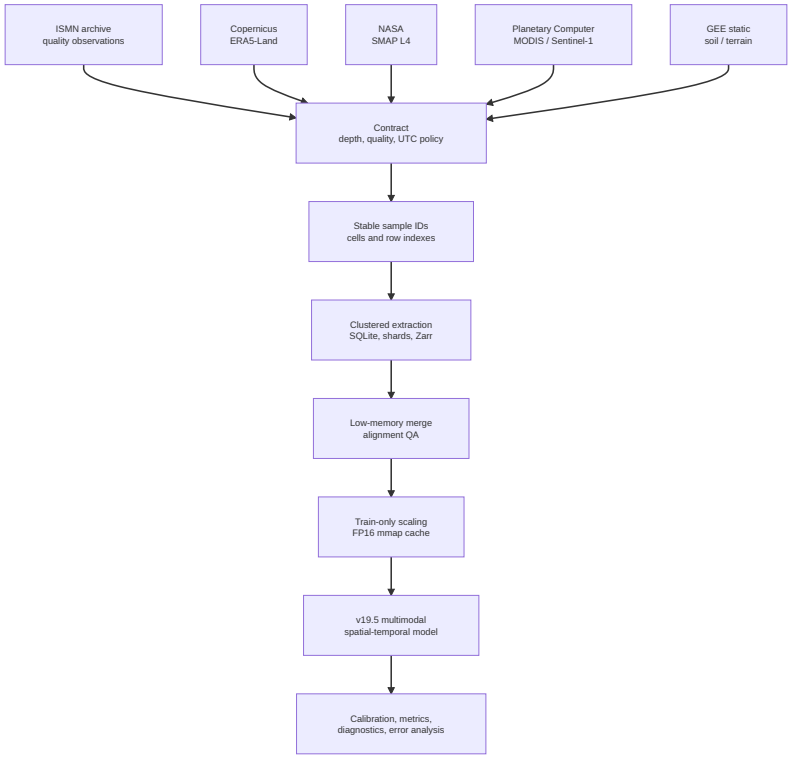


Figure 2.1: The global system is a chain of scientific contracts and computing systems, not only a neural-network block. Each stage reduces ambiguity before the next stage increases computational complexity.

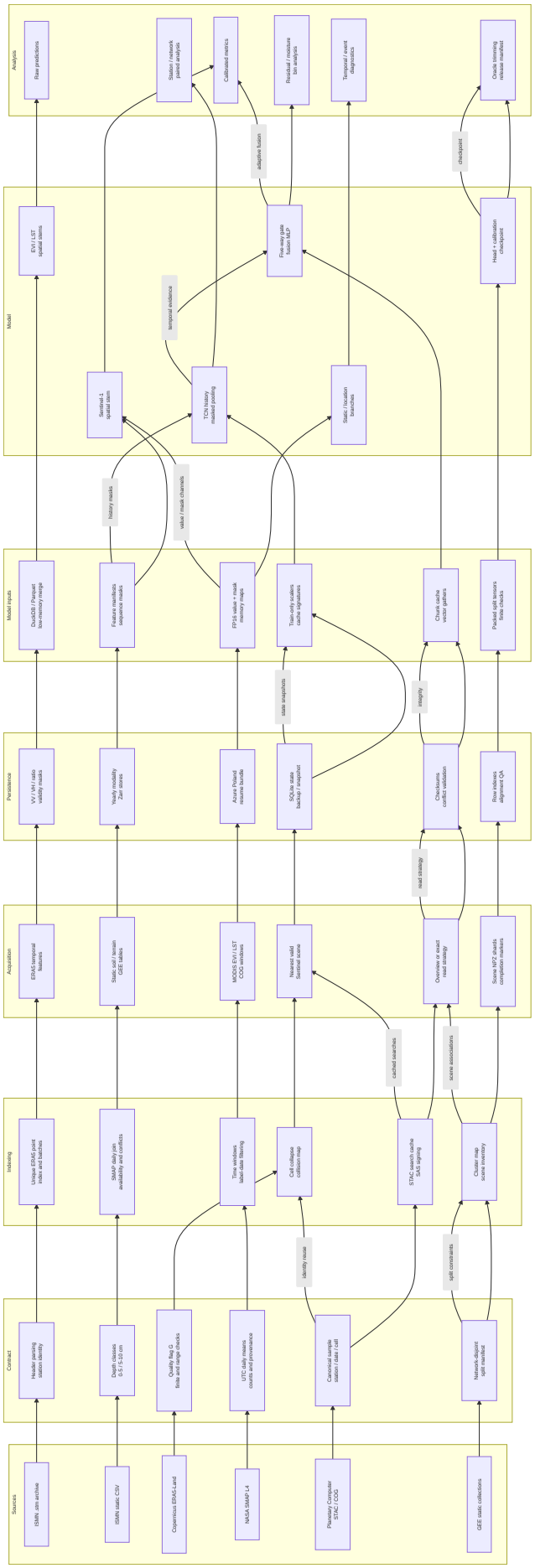


Figure 2.2: Full expanded production DAG. The diagram deliberately exposes the intermediate operations that are normally hidden inside broad pipeline boxes.

2.2 Scale of the project

The 143,850-line implementation is distributed across executable notebooks, Python modules, shell orchestration, validation programs, model exports, and analysis outputs. The number is useful as a description of scope rather than as a quality score: the repository contains separate systems for scientific contracts, observation parsing, gridded environmental retrieval, STAC/COG access, distributed recovery, storage conversion, cache construction, model training, and evaluation.

The scale is visible in the work products. The project audited 144 million observations, reduced them to more than 3.5 million unique station-date identities, indexed 204,033 Sentinel-1 scenes, associated those scenes with 528,280 cluster records, and produced 21,655 resumable shards. The final evaluation then operated over hundreds of thousands of held-out predictions. Each count corresponds to a different systems problem: streaming and audit, identity preservation, remote request amortisation, restartability, and full-population analysis.

Chapter 3

Global Data Contract and Ground Truth

3.1 Why a contract is necessary

When data arrive from dozens of networks and products, a model can train successfully on an invalid mixture. A contract makes the physical meaning explicit before storage and model code. The current global contract records UTC dates, a nominal 1 km grid, a 13×13 footprint, target units, accepted quality, depth classes, and channel counts.

3.2 Depth and quality

The target policy distinguishes near-surface 0–5 cm measurements from the explicit 5–10 cm fallback and excludes measurements outside 0–10 cm. Only quality flag `G` contributes to daily labels. This matters because a numerically finite value from a deeper sensor is not interchangeable with a surface value.

3.3 Archive audit and streaming conversion

The archive contains 4,340 station files, 4,002 compatible-depth files, and 61 networks. The converter supports directories and ZIP archives without extracting the complete archive. One source file is opened, its header is parsed, valid observations are accumulated by UTC day, and rows are written to Zstandard-compressed Parquet.

Listing 3.1: Conceptual streaming reduction used by the global converter

```
for source_file in iter_source_files(source):
    header = parse_header(source_file)
```

```
    if not compatible_depth(header):
        continue
    by_day = defaultdict(list)
    for fields in source_file.observations:
        if fields.quality != "G":
            continue
        day, value = parse_finite_measurement(fields)
        by_day[day].append(value)
    rows = daily_means_with_provenance(header, by_day)
    parquet_writer.write_table(rows)
```

The achievement is not just compression. Streaming changes the maximum working set from "the entire archive plus dataframe overhead" to "one source file plus one daily accumulator." The resulting Parquet table remains columnar, compressed, auditable, and joinable.

3.4 Ground-truth statistics

- 144,466,562 observations audited.
- 103,504,765 good-quality rows.
- 40,961,797 non-good rows.
- Zero malformed rows in the reported global profile.
- 5,045,849 daily Parquet rows.
- 2,210 stations and 61 networks in the daily profile.
- 3,514,643 unique station-date samples.

The headline benchmark combines primary and fallback depth classes. Depth-stratified reporting remains important because the hydrological response time and storage physics are not identical between 0–5 and 5–10 cm.

Chapter 4

Data Sources, Provenance, and Environmental Predictors

The predictors are introduced in the order in which they contribute to the physical interpretation of the target. Meteorological forcing describes the conditions that drive change, microwave observations provide a direct but coarse moisture-related anchor, optical and radar products add local spatial context, and static layers describe persistent differences between locations. Keeping these roles distinct helps explain both the value of fusion and the limits of treating every input as interchangeable.

4.1 Data sources and providers

The global dataset combines observations and environmental products produced or maintained by different scientific organisations. Keeping the provider explicit is important because the organisation that hosts a file is not always the organisation that created the measurement or product.

- **International Soil Moisture Network (ISMN):** in-situ soil-moisture observations and quality flags. The ISMN is maintained as an international network, with measurements contributed by individual monitoring networks and station operators.
- **ERA5-Land:** meteorological reanalysis, including precipitation and temperature fields. It is produced by the Copernicus Climate Change Service and the European Centre for Medium-Range Weather Forecasts (ECMWF).
- **Soil Moisture Active Passive (SMAP):** coarse-resolution microwave soil-moisture observations and the Level-4 soil-moisture product. SMAP is a NASA mission and its products are produced by NASA.

- **MODIS:** vegetation and land-surface-temperature products. MODIS is an instrument operated by NASA aboard the Terra and Aqua satellites; the products are produced by NASA.
- **Sentinel-1 RTC:** radar backscatter products used for VV and VH features. Sentinel-1 is an Earth-observation mission of the European Space Agency (ESA). The cloud-hosted assets used here are accessed through Microsoft Planetary Computer.
- **Microsoft Planetary Computer:** the Microsoft-hosted STAC catalogue and cloud storage interface used to discover and retrieve MODIS and Sentinel-1 assets. It is an access and hosting platform, not the scientific producer of those satellite missions.
- **Google Earth Engine:** the Google geospatial-computing platform used to extract the static soil and terrain layers. The platform provides the processing environment; the underlying products retain their original producers.
- **SoilGrids:** global soil-property maps used for clay, sand, and silt information. SoilGrids is produced by ISRIC—World Soil Information.
- **NASADEM:** elevation data used to derive terrain variables such as slope and aspect. NASADEM is produced from NASA and partner satellite radar data and distributed through NASA's Earth-observation data systems.

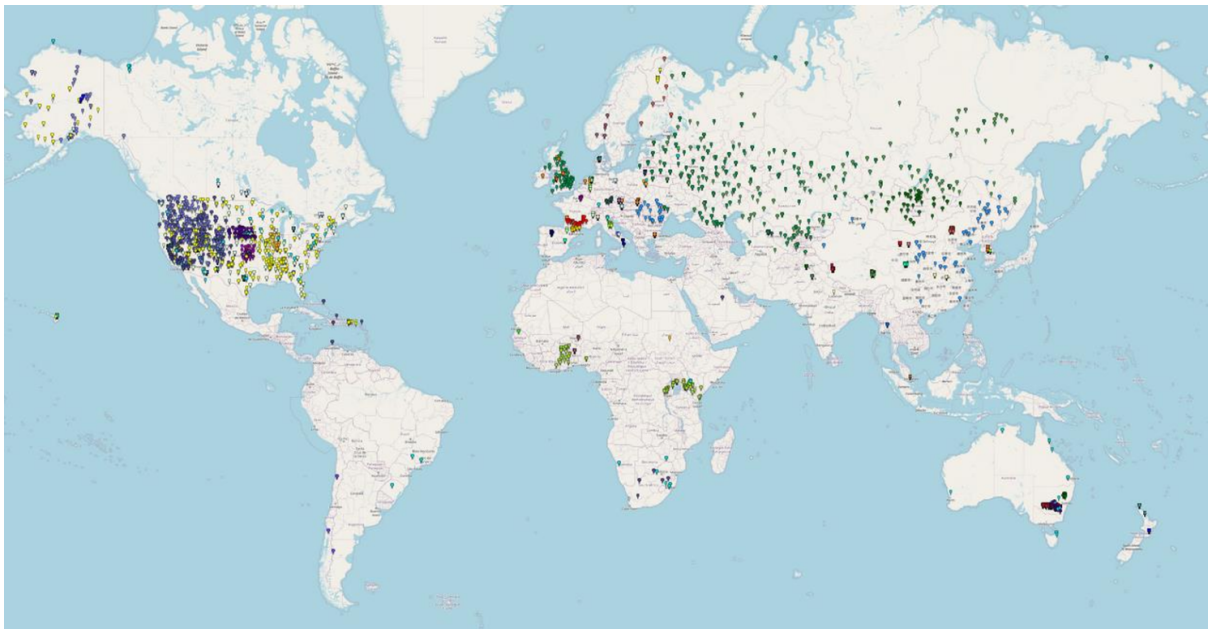


Figure 4.1: Global distribution of stations in the International Soil Moisture Network (ISMN), showing the worldwide observation network from which the global in-situ data are drawn. The map is reproduced as a reference to the ISMN network and the Austrian GCOS inventory report [GCOS Austria, 2025].

4.2 Provenance map

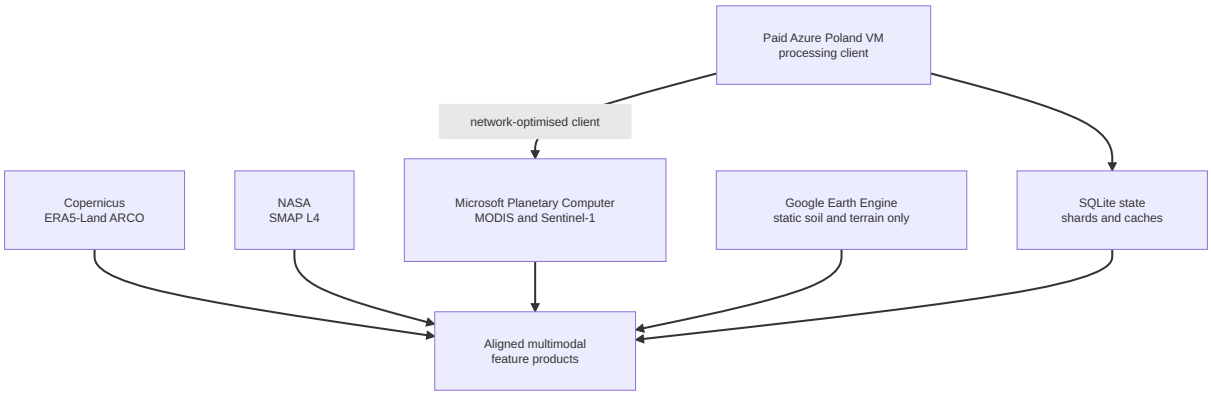


Figure 4.2: Data provenance and infrastructure roles. Azure accelerates processing; it does not replace Planetary Computer as the data host.

4.3 Copernicus ERA5-Land

ERA5-Land supplies daily meteorological forcing. The feature layer includes precipitation, 2-m temperature, dewpoint, vapour-pressure deficit, and an antecedent precipitation index. The access strategy reuses unique reanalysis grid points and processes point batches rather than repeatedly requesting the same grid location for nearby stations.

4.4 NASA SMAP

SMAP provides a broad-footprint microwave soil-moisture context. The extraction aggregates valid observations to daily station or cell values and retains observation counts, conflicts, and availability. Missing SMAP is represented explicitly rather than filled with a physical-looking zero.

4.5 MODIS from Planetary Computer

MODIS supplies EVI and LST through STAC discovery and COG or tile access. Year-partitioned Zarr arrays preserve the multidimensional product while keeping the scalar sample index separate. The EVI and LST arrays retain their distinct footprints rather than being forced into a false common native resolution.

4.6 Sentinel-1 RTC from Planetary Computer

Sentinel-1 provides radar-sensitive VV and VH context and their ratio. The nearest valid scene is selected within an 18-day tolerance, with deterministic earlier-date tie breaking. The production path uses a benchmarked 512 by 512 overview to approximate the 1 km target while retaining a separate exact native-resolution route.

Remote-sensing extraction is request-level scientific engineering. STAC searches, SAS signing, COG windows, overview semantics, acquisition age, cluster mapping, shard completion, and checksums all contribute to the numerical meaning of the final feature.

4.7 Static layers from Google Earth Engine

GEE contributes static soil and terrain variables: clay, sand, silt, elevation, slope, and aspect. Static extraction is separated from dynamic acquisition and reused across years. Older GEE notebooks are treated as experiments or prototypes, not as evidence that dynamic products came from GEE.

Chapter 5

Remote Acquisition and Distributed Data Engineering

Once the labels and predictor roles were defined, the next challenge was to obtain the data without losing either provenance or operational control. The remote products are large, intermittently accessed, and hosted through different interfaces. The following sections therefore describe not only where the data came from, but also how the extraction was organised so that a temporary network or machine failure did not become a scientific failure.

5.1 Why remote data extraction became the dominant systems problem

Remote raster extraction has fixed costs beyond pixel count: catalogue search, authentication and signing, TLS and HTTP setup, COG tile lookup, decompression, window conversion, reprojection, and local serialization. A design that performs one complete sequence for every station-date sample can be dominated by fixed overhead.

5.2 Cell reduction and collision maps

Stations are mapped to deterministic approximate 1 km cells. Collocated requests are collapsed for the expensive raster operation while a collision map retains the relationship between original station IDs and the representative cell. This is a major reduction in redundant work, but the collision map prevents the reduction from becoming an undocumented loss of identity.

5.3 Scene clustering

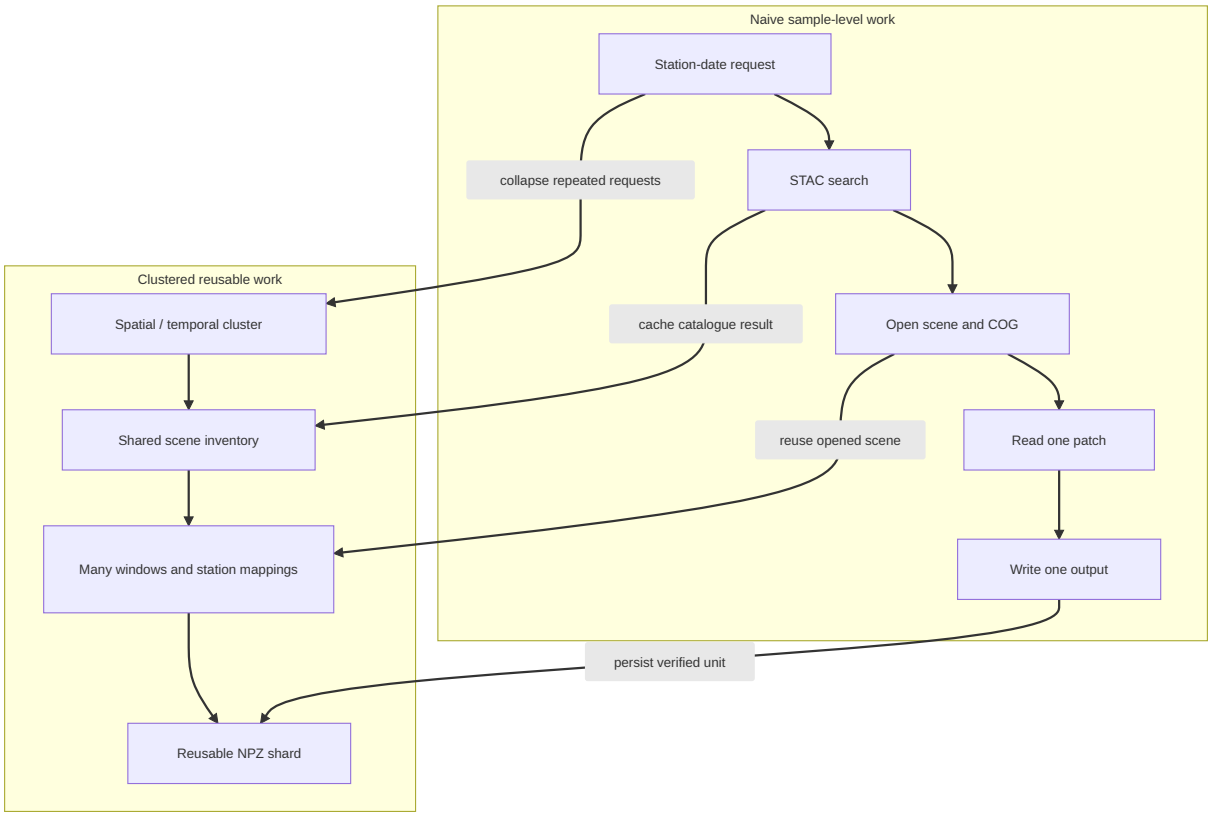


Figure 5.1: The clustering transformation changes the unit of work from a sample to a reusable spatial/temporal cluster.

The audited Sentinel state contained 819 cluster searches, 204,033 unique scenes, 528,280 scene-cluster associations, 818 clusters with associations, 21,655 NPZ shards, and no unlinked scenes. These are not merely bookkeeping numbers. They demonstrate that the remote archive was represented as a recoverable graph of scenes, clusters, and samples.

5.4 Azure Poland processing client

The Planetary Computer archive remained authoritative. After network-read optimisation, a paid Azure VM located in Poland served as the high-throughput client. The VM supplied a better-connected execution environment and adequate disk and memory for concurrent COG work. It was not a replacement archive.

5.5 Resume engineering

The workflow uses SQLite backup operations, cluster search state, scene shards, completion markers, manifests, and SHA-256 checksums. A failure therefore invalidates incomplete units rather than erasing all completed work.

This is analogous to checkpointing a long numerical simulation. A global remote extraction is decomposed into verified subproblems, and the computation can resume from the last trusted state. The same principle is later applied to local caches and model checkpoints.

5.6 Overview versus exact reads

The 1 km target does not necessarily require repeatedly downloading every 10-metre Sentinel-1 pixel. Overview reads reduce the remote payload and decompression cost. Exact reads remain available for validation and sensitivity analysis. Separate namespaces prevent approximate and exact arrays from sharing a filename and silently changing scientific meaning.

Chapter 6

Storage Formats, Merge, and Alignment

The extraction stage produces several kinds of information: scalar tables, mutable job state, multidimensional rasters, and completed evidence products. These objects have different access patterns, so the storage design is part of the scientific method rather than an implementation detail. The sections below explain how each representation is used and how the joins between them are checked.

6.1 Why no single file format is sufficient

The project uses each representation for a distinct access pattern:

- ZIP streaming for the compressed observation archive;
- Parquet for compressed, columnar scalar tables;
- SQLite for indexed mutable scene state;
- NPZ for completed transferable scene shards;
- Zarr for chunked multidimensional arrays;
- NumPy memory maps for local random access during training;
- JSON and CSV for human-readable manifests and provenance.

Choosing a universal format would force a compromise between streaming text, indexed state, cloud rasters, and GPU batch reads. The storage architecture is therefore an important part of the scientific design.

6.2 Low-memory merge

The merge path retains raster arrays in yearly modality-specific Zarr stores and joins scalar tables through stable sample identities and explicit row-index columns. DuckDB and Parquet scans avoid loading the complete global table into one pandas object.

6.3 The row-order hazard

A row-order error is more dangerous than a visible exception. If a Sentinel array has the correct shape but its sidecar IDs are in a different producer order, the model can train on plausible but incorrect pairs. The alignment checks therefore inspect row counts, sidecar IDs, producer sort contracts, expected patch shapes, metadata, completion flags, and sampled values.

Listing 6.1: Conceptual row-association validation

```
assert len(sidecar) == zarr_array.shape[0]
assert sidecar["sample_id"].is_unique
assert np.array_equal(sidecar["row_index"], np.arange(len(sidecar)))
assert finite(sampled_rows)
assert expected_patch_shape(zarr_array)
```

Index alignment establishes association, not geophysical equivalence. CRS, affine transform, pixel-centre offsets, overview resampling, and physical support area still require modality-specific geolocation checks.

Chapter 7

The Optimization Programme

The optimisation programme should be read as a sequence of responses to specific failure modes. The aim was not to make every individual operation faster in isolation. Instead, the design reduced unnecessary work, moved repeated work to an earlier stage, kept the CPU and GPU supplied at a controlled rate, and made failure recoverable. This distinction matters because a global experiment can be slow even when its arithmetic is efficient: the dominant cost may be waiting for data, moving data, or repeating work that has already been completed.

7.1 The counterfactual

The counterfactual makes the need for these choices concrete. Without optimisation, repeated remote reads, expanded patch tables, incompatible intermediates, per-batch raster decoding, and GPU starvation were estimated to produce approximately 700 GB of data and 60 days of top-tier GPU compute. The network transfer requirement would also be impractical. More importantly, this would have been a fragile calculation: a failure late in the run could have forced large portions of the earlier work to be repeated.

7.2 Optimization principle one: do not create data that will be discarded

The first decision was to prevent unnecessary data from entering the expensive part of the pipeline. Quality and depth filtering occur before expensive joins, so rejected observations do not trigger remote extraction. Daily labels replace redundant intraday rows for the model target while preserving their provenance. ERA5 requests are reused at unique grid points, static layers are exported once, and label-date manifests restrict dynamic

extraction to dates for which a target exists. Finally, only the active Sentinel-1 bands enter the production model cache. Each of these choices is small by itself, but together they reduce the number of rows, dates, bytes, and conversions that every later stage must handle.

7.3 Optimization principle two: amortize remote fixed costs

Remote operations have a fixed cost before any useful pixel is returned: connection setup, authentication, catalogue search, asset opening, and block decompression. Scene clustering, cached STAC searches, COG-aligned windows, overview reads, and shared scene opens amortise those costs over many station-date rows. The Azure location then makes the resulting request pattern practical by reducing the network distance and improving sustained transfer.

7.4 Optimization principle three: cache at the storage grain

The most useful cache unit is the unit the storage system can retrieve efficiently. Individual patch caches had little reuse because the unique/requested index ratio was close to one: many requested patches touched different source locations. The system instead caches native source chunks, decodes each touched chunk once, normalises it once, and gathers requested rows in batch order. This avoids paying the decompression and conversion cost again for every training example.

7.5 Optimization principle four: move deterministic work off the GPU

The GPU should spend its limited time on operations that benefit from massive parallel arithmetic. The local normalised cache therefore performs fill handling, physical scale conversion, validity detection, training-only standardisation, clipping, and FP16 conversion once. Training can then read value and mask channels from memory-mapped arrays rather than repeatedly asking the GPU to perform the same preparation for every epoch.

7.6 Optimization principle five: bound parallelism

Parallelism is useful only while the shared resources can support it. Unbounded workers and prefetch queues can exhaust RAM, thrash the disk cache, or trigger remote throttling. The training system therefore caps workers, prefetch depth, and future-batch queues. Pinned memory and asynchronous CUDA copies are used within that bounded system so that short delays can be hidden without turning the input pipeline into an uncontrolled memory consumer.

7.7 Optimization principle six: preserve the effective batch under finite VRAM

The desired effective batch was 2,048, but the available VRAM could not hold all of those examples and their intermediate activations in one allocation. Microbatches and gradient accumulation preserve the effective batch without requiring that single allocation. Automatic mixed precision reduces the memory occupied by suitable intermediate values, while gradient clipping, EMA, and best-checkpoint restoration improve stability when a long run encounters unusually difficult batches or noisy epochs.

7.8 Optimization principle seven: make caches self-describing

An optimisation is dangerous if it makes stale data look current. Scaler hashes and build signatures prevent arrays generated under different decode rules, years, or normalisation settings from being mixed. Temporary `.building` files are promoted only after validation, so an interrupted construction cannot silently replace the last complete cache.

7.9 Optimization register in prose

The engineering decisions are interdependent. Streaming ingestion reduces host memory; Parquet reduces persistent scalar volume; clustered extraction reduces remote fixed costs; Zarr preserves spatial arrays; chunk caching reduces decompression; normalised memory maps reduce CPU work; bounded prefetch reduces GPU idle time; and gradient accumulation preserves optimisation behaviour under VRAM limits. Treating these as isolated tricks would miss their multiplicative effect.

7.10 What the optimisation means in computer terms

For a physics reader, the central distinction is between computation and movement. A processor may be able to calculate a mean over a raster window very quickly, but it cannot calculate until the bytes have arrived from the remote server, been authenticated, decompressed, copied into memory, and arranged in the expected array. In the original station-date formulation, those fixed steps were repeated for many rows that shared the same scene or grid cell. Clustering changes the accounting: one expensive remote read produces a reusable object, and many scientific rows point to that object through an index.

The network optimisation therefore works at three levels. First, the catalogue search is cached so the same question about available scenes is not sent again. Second, nearby windows are grouped so one connection and one scene open can serve several requests. Third, COG overviews select a lower-resolution internal tile when the target is approximately one kilometre, avoiding transfer of unneeded ten-metre pixels. The third step is not free: it changes the support of the measurement. That is why the exact path is retained, the strategy is named in the manifest, and exact-versus-overview comparisons remain a required validation.

The storage optimisation addresses a different resource: random access. A compressed archive is efficient for distribution but poor for repeatedly extracting arbitrary rows. A full dense array is easy to index but can exceed RAM and takes too long to rebuild after a failure. Chunked Zarr arrays reduce the amount read from disk; memory-mapped arrays let the operating system bring only requested pages into memory; and precomputed normalisation means the training loop does not repeatedly perform the same conversion. The validity mask is stored beside the value because replacing an absent value with a numeric fill would make missingness indistinguishable from a physical state.

The CPU and GPU optimisations address a pipeline-balance problem. If the CPU prepares batches more slowly than the GPU consumes them, the GPU waits. If workers prepare too far ahead, RAM fills with batches that may never be used. Bounded worker counts and prefetch queues keep the queue between those two failure modes. Pinned memory and asynchronous copies overlap host-to-device transfer with arithmetic where possible. Mixed precision reduces the size of activations, while accumulation allows many small device batches to produce the gradient statistics of a larger effective batch. These changes do not mean the model performs fewer conceptual updates; they change when bytes move and when the optimiser commits an update.

Finally, restartability is a time optimisation as much as a reliability feature. A 700 GB-style run is not one indivisible calculation; it is a collection of searches, scenes, chunks, shards, and manifests. A completion marker is a promise that a particular unit passed its

checks. After an interruption, the scheduler can spend time only on units without a valid promise. Checksums prevent a truncated file from being mistaken for a completed unit, and cache signatures prevent an old unit produced under different settings from being reused. The result is fewer wasted hours, fewer repeated network transfers, and a more interpretable scientific run.

The value of the optimisation programme is not a single speedup. It is that an experiment which could fail for storage, network, memory, or restart reasons became an experiment whose assumptions are explicit and whose intermediate states can be verified.

7.11 Why the unoptimised experiment was not practical

It is useful to describe the original difficulty as a queue of physical operations rather than as an abstract "large dataset". A request began at a station and date, contacted a catalogue service, obtained an authorised asset URL, opened a remote raster, downloaded compressed blocks, decompressed them, converted their numeric encoding to physical units, handled missing pixels, assembled a patch, and finally wrote the result. Only after all of those steps could one small training row be produced.

The same scene, grid cell, or source chunk was often needed by many rows. If each row repeated the complete sequence, the experiment paid the fixed costs again and again. The expensive part was therefore not only the arithmetic needed for the final mean or neural-network prediction. It was the movement of data: network latency, authentication, transfer, decompression, disk reads, memory copies, and conversion between representations. A faster processor cannot remove time spent waiting for a remote server or a disk.

7.11.1 A simple analogy for the processing pipeline

The computer can be viewed as a factory with four work areas. The remote archive is the supplier, the CPU is the preparation area, the GPU is the specialised high-speed machine, and the disk is the warehouse. The GPU is very fast at tensor arithmetic but has limited local memory. If the supplier sends parts one at a time, the GPU stands idle. If preparation sends parts faster than the GPU can consume them, the warehouse fills and the machine may run out of memory. A successful design must keep all four areas busy without allowing any one queue to become unbounded.

This analogy also explains why several modest optimisations were needed together. Clustering improved the supplier connection, chunk caching reduced warehouse retrieval, precomputed normalisation reduced preparation, bounded prefetch controlled the queue,

and microbatching adapted the work to the GPU's limited local memory. None of these changes alone solves the complete problem.

7.11.2 Network traffic: reduce requests before reducing workers

Adding more workers to a poorly organised remote workload would have made the problem worse. Every worker would have repeated catalogue searches, opened the same scenes, and competed for the same network and remote service limits. The pipeline first changed the unit of work. Station dates were grouped into canonical cells and spatial/temporal scene clusters. One search result and one opened source scene could then serve many station-date rows.

The STAC search cache records the answer to a catalogue question. The scene inventory records which source scenes can serve which clusters. Signed asset URLs and request metadata are retained so a retry does not have to rediscover the entire remote state. This changes the workload from "ask the archive for every row" to "discover the reusable source once, then reference it by index". The scientific identity remains the station-date row; only the computer's route to the data changes.

7.11.3 Why overview reads helped, and what they changed

Sentinel-1 native pixels are much smaller than the approximately one-kilometre model target. Reading every native pixel for every sample would transfer and decompress information that would later be averaged away. Cloud-Optimised GeoTIFF overviews provide precomputed lower-resolution representations, so the remote read can request a smaller block when that support is scientifically appropriate.

This is a performance approximation, not a free replacement for the source. An overview represents a different spatial support and may use a different resampling path. The extraction therefore records whether an overview or exact native read was used, keeps exact reads available for validation, and prevents the two strategies from sharing an ambiguous filename. The optimisation saves network bytes and decompression work while preserving the information needed to quantify the approximation.

7.11.4 Memory: why a correct calculation can still crash

Memory is not one single pool. The GPU has fast but limited VRAM; the CPU has larger system RAM; and the operating system uses disk-backed pages when RAM is under pressure. A batch can fail even when the final model is small because the batch also

requires intermediate activations, gradients, temporary arrays, worker queues, and copies between host and device.

The production cache avoids constructing the entire global tensor in RAM. Zarr stores multidimensional arrays in chunks, so a request loads the touched chunks rather than the whole year. Memory maps expose large prepared arrays as file-backed regions; the operating system brings in the pages actually used by the next batch. A value channel is stored beside a validity channel, so a missing pixel is not confused with a genuine physical zero or fill value.

Normalisation, clipping, fill handling, and conversion to the training precision are performed once while the cache is built. Without this step, the same deterministic work would be repeated for every epoch. The training loop then performs the work that must change sampling, forward prediction, backpropagation, and optimisation instead of repeatedly decoding the source archive.

7.11.5 GPU memory: microbatches preserve the learning problem

The desired optimisation batch was 2,048 examples, but placing 2,048 complete examples and all their intermediate tensors on the GPU at once would exceed VRAM. The solution was not to change the intended batch arbitrarily. A microbatch that fits in VRAM is processed, its gradients are accumulated, and several microbatches are combined before one optimiser update. Thus the physical batch controls memory while the effective batch preserves the intended update scale.

Automatic mixed precision stores many intermediate values in a smaller floating-point representation when safe, reducing memory traffic and often accelerating tensor operations. Gradient clipping limits the effect of an exceptionally large update. Exponential moving averages provide a smoother parameter state for checkpoint selection. These mechanisms address different failure modes: VRAM pressure, numerical instability, and sensitivity to the last few noisy updates.

7.11.6 CPU GPU balance: keeping the expensive machine occupied

The GPU cannot start a batch until the CPU has gathered the requested rows, assembled temporal history, applied masks, and copied the result to device memory. If this preparation takes longer than the GPU computation, utilisation falls even though the GPU itself is capable of much more work. Conversely, unlimited workers can prepare too many batches,

consume RAM, and cause contention that makes every worker slower.

The data loader therefore uses a bounded number of workers and a bounded prefetch queue. Pinned host memory makes transfers more predictable, and asynchronous copies can overlap transfer with computation where the hardware allows it. The queue is deliberately bounded: it is large enough to hide short storage delays but small enough that an unused future batch cannot exhaust memory. This is a control problem, not simply a request to maximise the worker count.

7.11.7 Storage formats: each format removes a different bottleneck

The project uses several formats because the operations are different. ZIP is useful for distributing the original archive but poor for random row access. Parquet is efficient for columnar tables and low-memory joins. SQLite provides indexed mutable state for scene searches and completion records. Zarr supports chunked raster arrays. NPZ shards make completed remote units portable and independently checkable. Memory-mapped NumPy arrays provide fixed-offset training access.

Converting everything into one format would move the bottleneck rather than remove it. The important engineering rule is that every conversion preserves a mapping back to the canonical sample identity. Shapes alone are insufficient: an array with the right dimensions can still contain the wrong rows. Sidecar row indexes, manifests, finite-value checks, completion markers, and checksums make the cheaper representation auditable.

7.11.8 Restartability: saving time when failure is expected

At global scale, interruption is a normal operating condition. A network timeout, expired asset, worker failure, full disk, or machine restart should not force the project to repeat all previous work. The extraction is therefore divided into independently committed units such as catalogue searches, scene clusters, source chunks, and shards.

An output is not considered complete merely because a filename exists. The workflow checks dimensions, source identifiers, valid-value counts, metadata, and checksums before recording completion. Writes use temporary paths and are promoted atomically only after validation. On restart, the scheduler trusts only units with a valid completion record and retries the rest. This converts a potentially all-or-nothing computation into a sequence of smaller verified computations.

7.11.9 The resulting saving in practical terms

The optimisations saved more than wall-clock time. They reduced repeated network transfer, reduced remote catalogue pressure, reduced peak RAM and VRAM, reduced disk churn, reduced GPU idle time, and reduced the amount of work lost after an interruption. They also made the result explainable: a scientist can identify which source asset, read strategy, cache signature, row mapping, and checkpoint produced a result.

Bottleneck	**Naive behaviour**	**Production response**
Remote archive	Repeat searches, authentication, scene opens, and downloads for individual rows	Cell reduction, scene clustering, cached searches, shared scene reads, and explicit overview strategy
CPU and disk	Repeated decompression, conversion, and random reads during every epoch	Chunked Zarr, source-chunk cache, normalised mmap arrays, and vectorised gathers
GPU memory	Attempt to place the full optimisation batch on limited VRAM	Microbatches, gradient accumulation, AMP, and batch-size probing
Host memory	Unlimited workers and prefetch create an unbounded queue	Bounded workers, bounded prefetch, pinned memory, and finite caches
Failures	One interruption invalidates a long monolithic run	Atomic shards, completion markers, manifests, checksums, and resume logic

Chapter 8

Engineering Internals of the Global Run

The preceding chapters described the pipeline from the outside. This chapter now follows the data through the computer itself: which unit is processed at each stage, how intermediate representations are stored, how a failed unit is distinguished from a completed one, and how the training loop moves data between storage, CPU memory, and GPU memory. This view connects the engineering decisions to the scientific guarantees reported later.

8.1 The unit of computation was redesigned several times

The first conceptual unit of the experiment was a station-date sample. That unit is scientifically natural because the target is defined at a station and date, but it is computationally inefficient. Many station dates fall in the same raster cell, request the same reanalysis grid point, or refer to the same remote scene. Treating every row as an independent download would repeat network requests, authentication, decompression, and array conversion.

The production system therefore used different units at different stages. A station-date remained the unit of scientific identity. A canonical cell became the unit of spatial request sharing. A scene-cluster became the unit of remote Sentinel work. A yearly Zarr chunk became the unit of persistent raster storage. A packed tensor row became the unit of training. This separation is important: it reduces repeated work without changing the definition of the target row. Each conversion writes an index that maps the cheaper computational unit back to the original station-date identity.

8.2 Storage layers and why no single format was sufficient

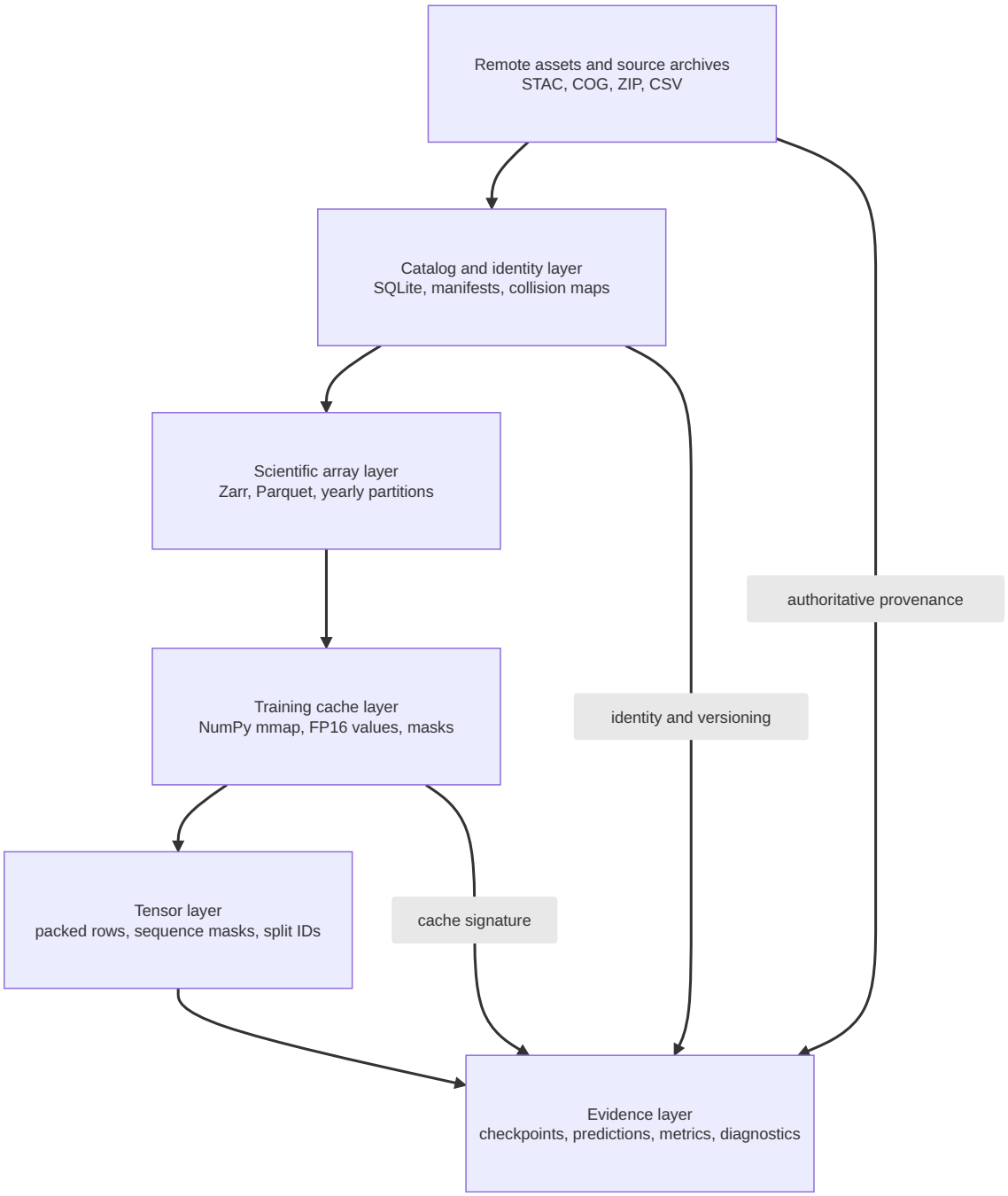


Figure 8.1: The storage stack separates authoritative inputs, scientific arrays, training representations, and evidence products.

Source formats were retained where they carried provenance. SQLite was used for small mutable state because it supports indexed lookup, transactions, and backup snapshots. Zarr was used for chunked multidimensional arrays because a single station-date read should not require loading an entire year. Parquet was used for tabular joins and audits

because columns can be read selectively. NumPy memory maps were used for training because they permit fixed-offset access without materialising all rows in RAM. NPZ shards were used for resumable remote extraction because a completed shard is independently verifiable and transferable.

The resulting design deliberately contains duplicate representations. That is not accidental storage waste: each representation is optimised for a different access pattern. The source layer is authoritative, the array layer is structured, the cache layer is fast, and the tensor layer is model-shaped. Hashes, manifests, shapes, and row-order sidecars are what make those representations equivalent rather than ambiguous copies.

8.3 Resumable extraction as a state machine

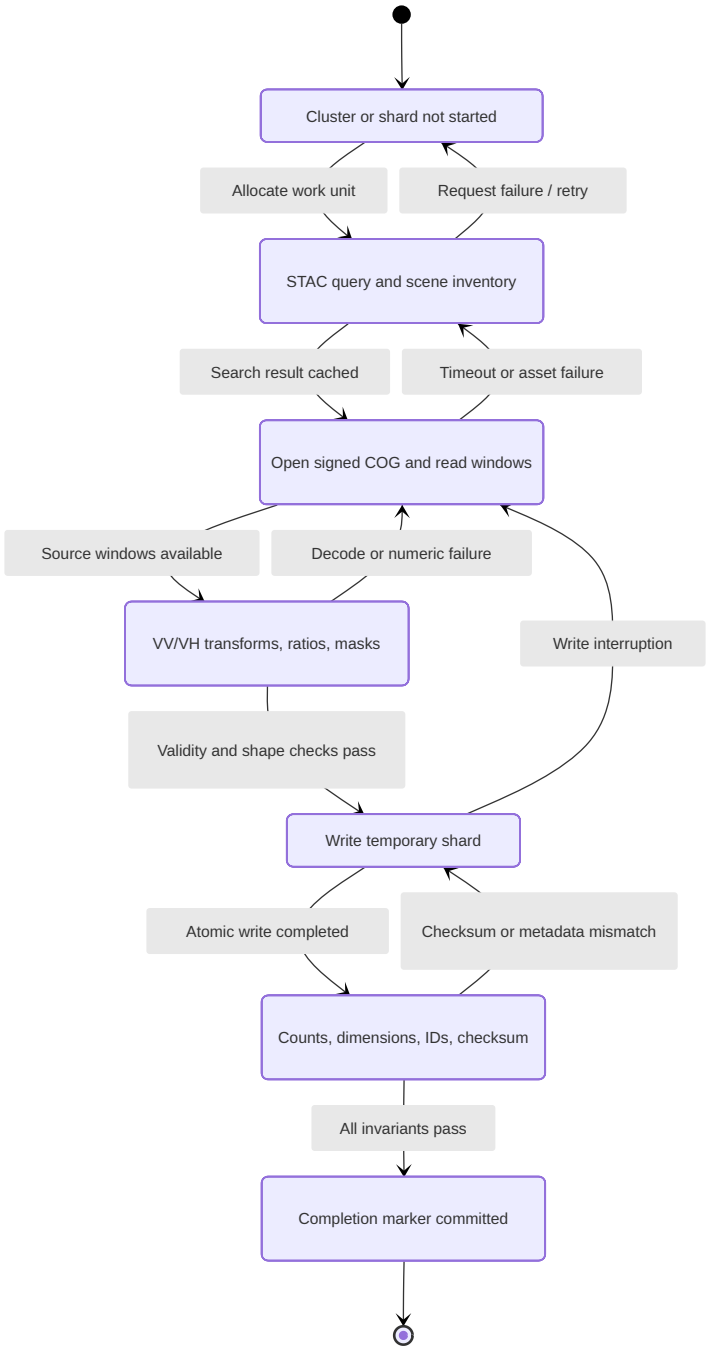


Figure 8.2: A failed remote operation returns to the last trusted state rather than invalidating the complete extraction.

The distinction between an attempted operation and a committed operation is central. A scene search can be repeated safely because its result is cached. An array write is not considered complete merely because a file exists: the workflow also records dimensions, valid-value counts, source identifiers, and a checksum. A partial write therefore cannot masquerade as a valid shard after a machine interruption. SQLite backup and snapshot

operations provide a second line of defence when the mutable state database itself must be moved between machines.

8.4 Request geometry, compression, and network cost

Remote raster access has at least four costs: request latency, bytes transferred, decompression, and local window assembly. A small request is not always cheaper because authentication and HTTP setup are paid per request. A large request is not always cheaper because it transfers pixels that will never enter a sample. The production extraction balanced these costs by clustering nearby requests, using COG overviews when the target resolution permitted it, and retaining exact native reads for validation.

The overview path changes the physical pixel support of the input. It is therefore represented as a named strategy rather than silently replacing the native product. The output records the strategy, source asset, requested window, valid-pixel count, and fill behaviour. This turns a performance approximation into an explicit experimental factor. A future ablation can compare exact and overview-derived features without reconstructing the entire archive.

8.5 Cache signatures and invalidation

Persistent caches are useful only if stale values cannot be confused with new values. Cache names and manifests include the contract version, source namespace, feature list, scaling policy, precision, clipping policy, and producer signature. A change to any of these fields produces a new cache identity. Temporary files are promoted atomically only after validation, so a process killed during construction leaves the previous complete cache intact.

This design also protects scientific interpretation. If a scaler is changed, the model input distribution has changed. If a fill value is changed, the missingness semantics have changed. If a raster row order is changed, the station-date join may be wrong even though every array has the expected shape. Signatures make such changes visible in the release manifest instead of leaving them as undocumented differences between runs.

8.6 Memory traffic during training

Training performance was constrained by memory movement as much as by matrix multiplication. The cache stores a value channel and a validity channel, uses FP16 for

the normalised local arrays, and gathers only the rows and temporal steps needed by a batch. Bounded workers and prefetch queues prevent the data loader from creating an unbounded backlog. Pinned host memory permits asynchronous transfer to the GPU where the framework and hardware support it.

Gradient accumulation separates the optimisation batch from the physical batch that fits in device memory. Several small forward/backward passes are summed before an optimiser update, approximating a larger batch without allocating all examples simultaneously. Mixed precision reduces activation storage, while gradient clipping prevents a rare large residual or invalid intermediate from destabilising an update. These mechanisms address different bottlenecks and cannot be treated as interchangeable speed settings.

Chapter 9

Model Architecture and Physical Interpretation

The model is introduced only after the data contract and computational pathway are established. This is deliberate: the architecture is a response to the available measurements and their missingness patterns, not an isolated choice of neural-network components. Each branch has a defined role, and the fusion stage combines those roles while retaining enough structure to analyse the result.

9.1 Final v19.5 configuration

- Architecture: multiscale residual spatial-temporal attention gated fusion.
- Spatial width: 48.
- Temporal hidden width: 64.
- Fusion width: 192.
- Eight history steps in a maximum 30-day window.
- AdamW, MSE in the final exported configuration.
- Effective batch size 2,048.
- EMA decay 0.85 and gradient clipping norm 2.0.

9.2 Full model diagram

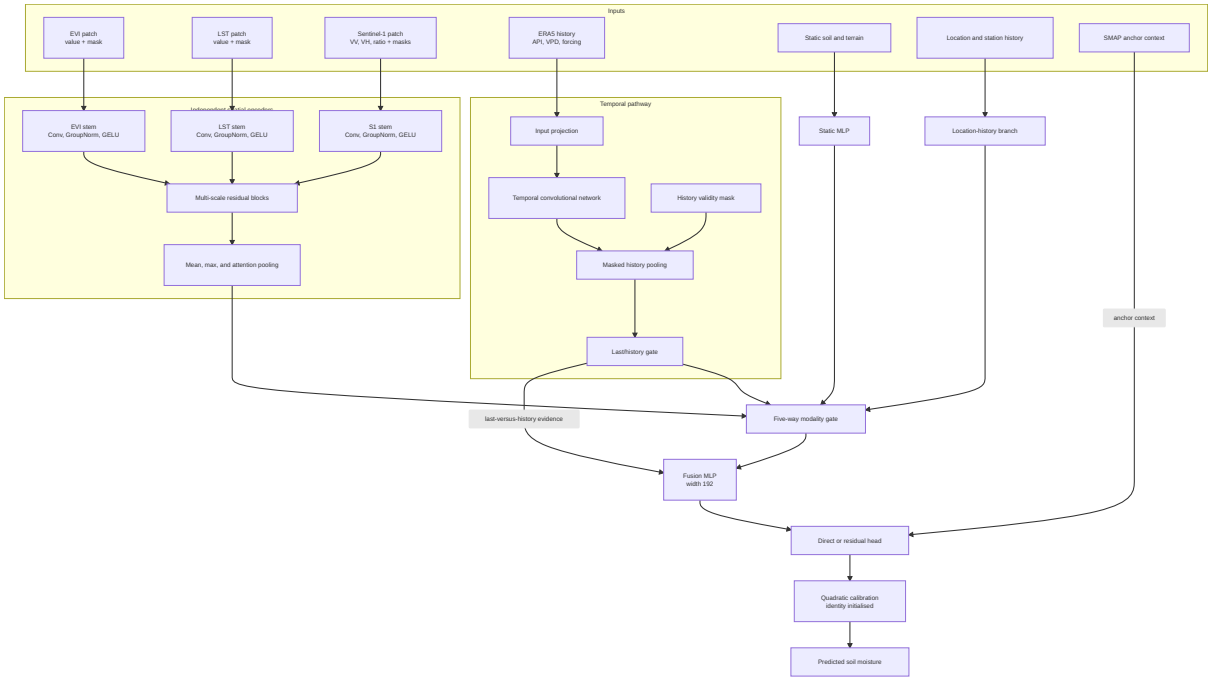


Figure 9.1: v19.5 model architecture. Separate branches preserve measurement semantics before adaptive fusion.

9.3 Spatial encoders

EVI, LST, and Sentinel-1 receive independent stems because their values have different units, noise, missingness, and physical interpretations. Each stem uses convolution, GroupNorm, GELU, and multi-scale residual blocks. A local depthwise 3×3 path captures nearby structure; a dilated path enlarges context; a 1×1 mixer combines channels; and the residual connection stabilises optimisation.

Mean pooling describes the average neighbourhood. Max pooling retains a salient extreme. Attention pooling learns which spatial positions matter for the current example. Concatenating all three avoids forcing one summary statistic to represent every landscape.

9.4 Temporal TCN

The temporal branch projects a sequence into hidden width, applies ordinary and dilated one-dimensional convolutions, scores valid history steps, and gates the last valid state against the broader history. This resembles a learned finite-memory filter: it can represent rainfall persistence, delayed infiltration, evaporation-driven drying, and recent-event dominance.

9.5 Static and location branches

Clay, sand, silt, elevation, slope, and circular aspect are projected through a static MLP. A location-history configuration also receives a controlled spatial representation. This can improve regime interpolation but creates a memorisation risk; a network-disjoint split is stronger than random rows but is not identical to a continent or biome holdout.

9.6 Modality gate and fusion

The five-way gate produces non-negative weights summing to one over EVI, LST, Sentinel-1, temporal history, and static context. These weights are predictive routing coefficients, not causal contribution fractions. The fusion MLP then models conditional interactions such as precipitation followed by high VPD on sandy soil or radar structure under dense vegetation.

9.7 Calibration

The calibration layer is:

$$\hat{y} = c_0 + c_1 a + c_2 r + c_3 ar + c_4 a^2 + c_5 r^2,$$

where a is the anchor and r is the raw model output or residual representation. Identity initialisation means the layer begins as a sensible no-correction map and learns only the correction supported by the data.

A calibration layer can improve output scale without adding new physical representation. The calibrated score belongs to the complete system, not necessarily to the uncalibrated neural encoder alone.

Chapter 10

Training, Numerical Stability, and Reproducibility

Training is the point at which the prepared data, model design, and hardware constraints meet. A model can be mathematically well specified and still produce an unreliable experiment if batches are too large for memory, if mixed-precision arithmetic is unstable, or if a checkpoint cannot be traced back to its input contract. These sections therefore describe the optimisation procedure together with the safeguards that make it repeatable.

10.1 Hyperparameter search

The project uses staged Optuna studies rather than fully training every candidate. Shorter and bounded trials screen architectures and optimisation settings before full retraining. The final exported configuration is treated as the authority when historical notebook defaults differ.

10.2 Loss and optimisation

The final exported v19.5 configuration records MSE:

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2.$$

AdamW updates the weights using gradients from backpropagation while weight decay discourages unnecessarily large parameters. Gradient clipping limits rare extreme updates.

10.3 AMP, accumulation, and EMA

Automatic mixed precision reduces memory and can accelerate matrix operations. Gradient accumulation approximates a large batch through several microbatches. EMA maintains a smoothed parameter state that is less sensitive to the noise of the final individual updates. These mechanisms improve feasibility but add state and numerical conditions that belong in the reproducibility record.

10.4 Compilation and hardware dependence

The v19.5 training path smoke-tests `torch.compile` with a reduce-overhead mode and falls back to eager execution when compilation is unstable. This is a practical engineering compromise: compilation can reduce framework overhead but must not be a single point of failure.

10.5 Checkpoint policy

Checkpoints are written atomically, and configuration, scaler, manifest, and hash information is retained. The historical workflow includes test-R^2 checkpoint selection; this is explicitly documented because repeated test selection means the nominal test set should be interpreted as development-held-out rather than perfectly untouched.

Chapter 11

Evaluation and Error Analysis

The evaluation is presented as a set of connected questions rather than as one headline number. First, the aggregate scores establish the overall level of agreement. The diagnostic analyses then ask where the errors occur, whether they change with time or moisture regime, and whether the apparent gain is consistent across stations. This order keeps the interpretation close to the measurement process and avoids treating a single average as a complete description of model behaviour.

11.1 Evaluation philosophy

Evaluation is treated as an observational campaign analysis. One global score cannot show whether errors are concentrated in a network, season, moisture bin, region, acquisition-age stratum, or missingness regime.

11.2 Primary results

Configuration	Rows	RMSE	MAE	R^2
Environment-only	644,716	0.08626	0.06794	0.47046
Location-history	905,248	0.06594	0.04854	0.65341

The location-history model wins at 62.62% of matched stations, with paired RMSE 0.06704 versus 0.08490 for the environment-only model. These paired statistics are more informative than comparing aggregate row counts alone.

11.3 Temporal and regional diagnostics

Year-by-year performance is strong in 2023–2024 and weaker in 2025–2026. This may reflect distribution shift, event composition, station mix, coverage, or target/raster availability. The analysis reports the pattern without claiming a causal explanation.

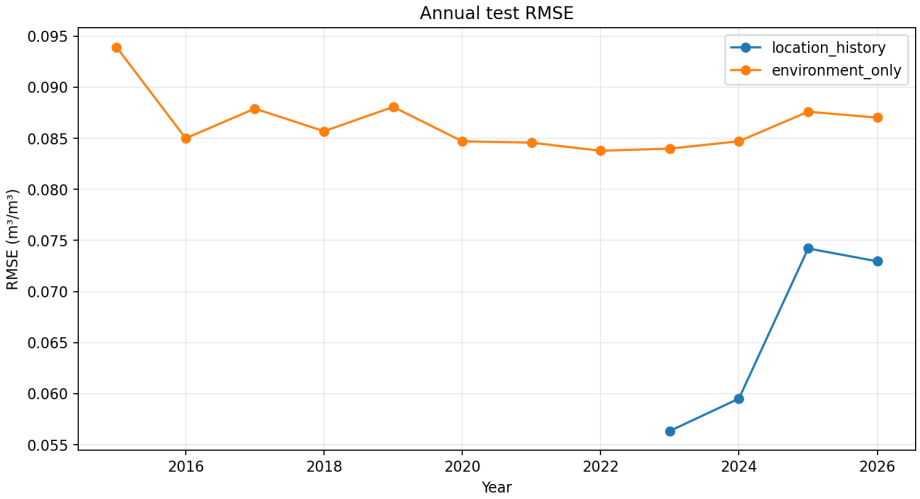


Figure 11.1: Annual test RMSE diagnostic. The temporal pattern is a monitoring signal, not by itself a causal explanation.

11.4 Figures from the global analysis

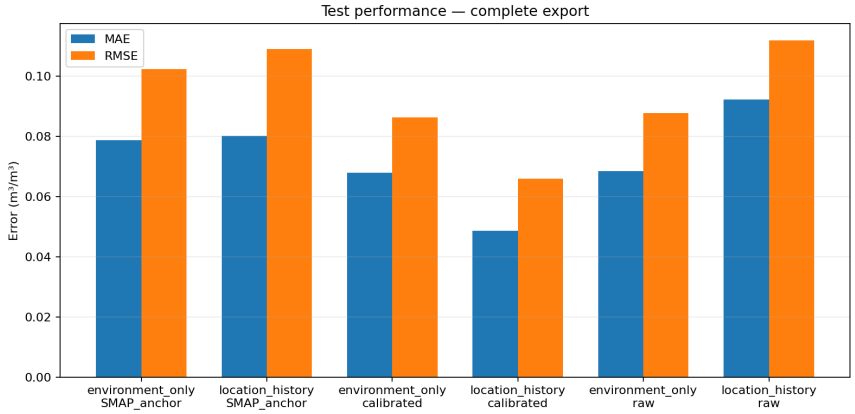


Figure 11.2: Overall held-out MAE and RMSE comparison.

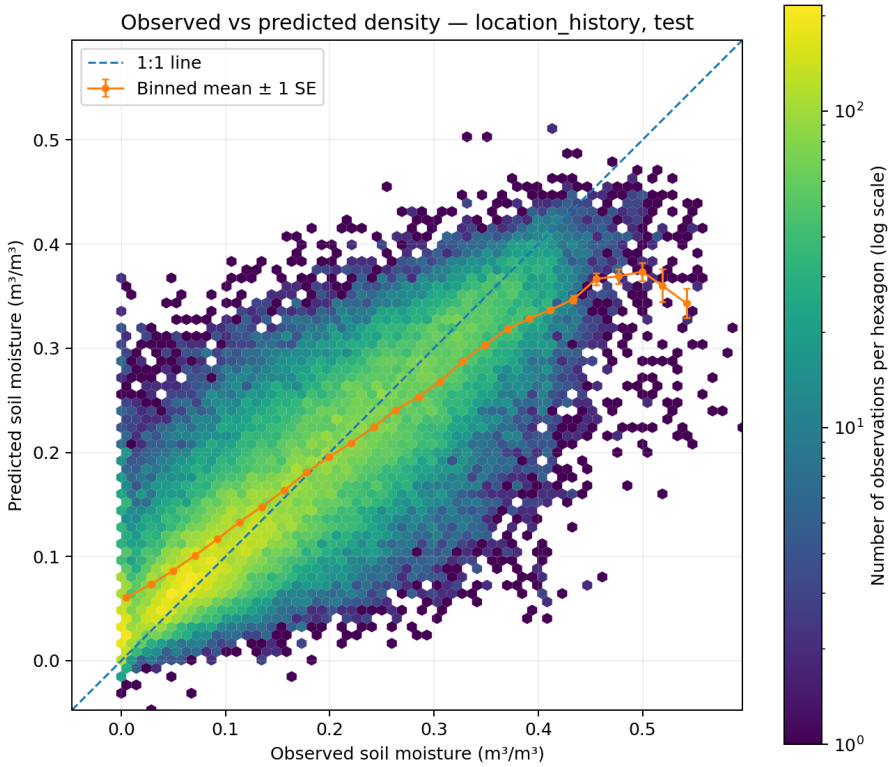


Figure 11.3: Observed versus predicted density for the location-history test export.

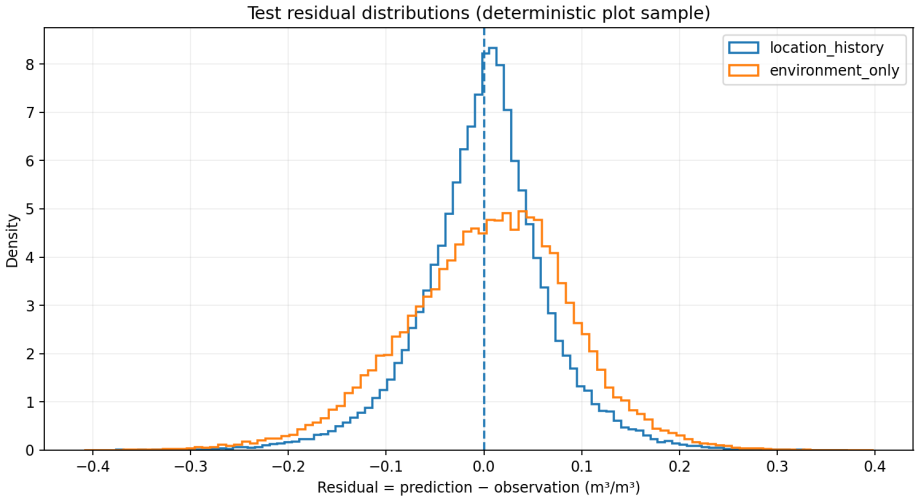


Figure 11.4: Residual distributions for the held-out configurations.

11.5 Oracle trimming

Oracle trimming ranks predictions by their realised error using the true target and recomputes retained-set R^2. The location-history model reaches $R^2 = 0.70$ after removing 1.30% of its worst errors, $R^2 = 0.80$ after 7.90%, and $R^2 = 0.90$ after 24.10%. The

environment-only model requires 16.80%, 32.30%, and 54.10%, respectively.

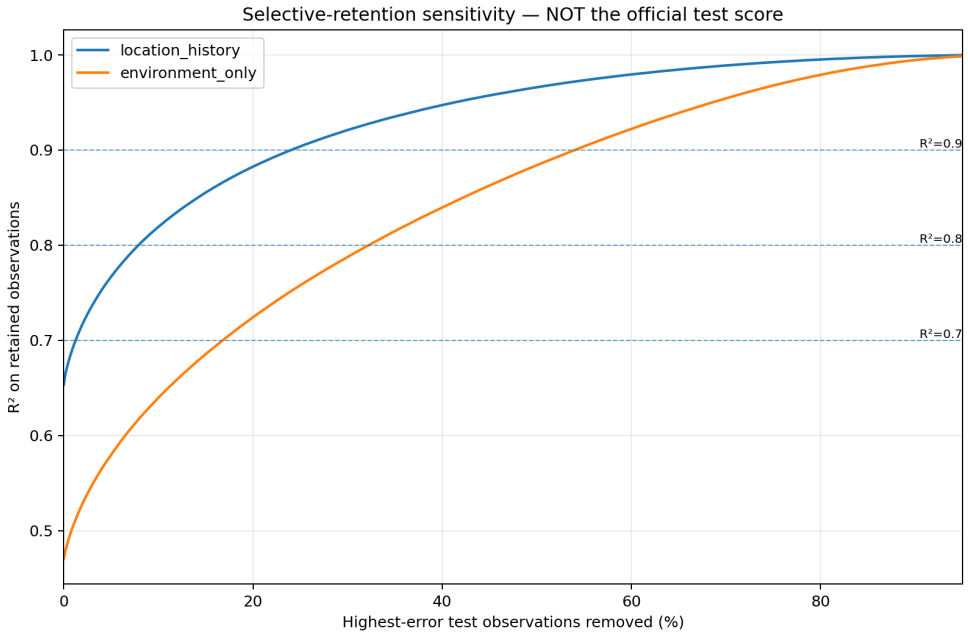


Figure 11.5: Oracle high-error trimming sensitivity. This is a diagnostic of error concentration, not an operational filtering method.

A deployed system cannot know the true target before prediction. Operational filtering must use information available before prediction, such as missingness, acquisition age, uncertainty, or regime flags.

Chapter 12

Extended Diagnostic Gallery and Technical Case Studies

12.1 Why the gallery is part of the result

The figures in this chapter are not decorative appendices. Each plot answers a different failure question. A global average tests aggregate agreement; a station-level diagnostic tests concentration across sites; a month-of-year plot tests seasonal dependence; a moisture-bin plot tests range compression; and a paired station comparison tests whether one configuration consistently helps the same locations; residual plots test whether calibration has removed scale-dependent bias.

12.2 Monthly and seasonal behaviour

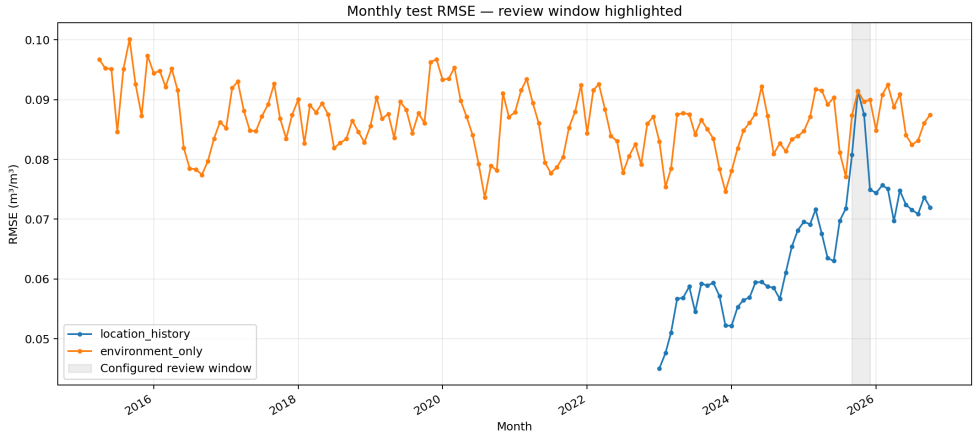


Figure 12.1: Monthly test RMSE. Seasonal error is a property of the joint observation and acquisition system, not automatically a model defect.

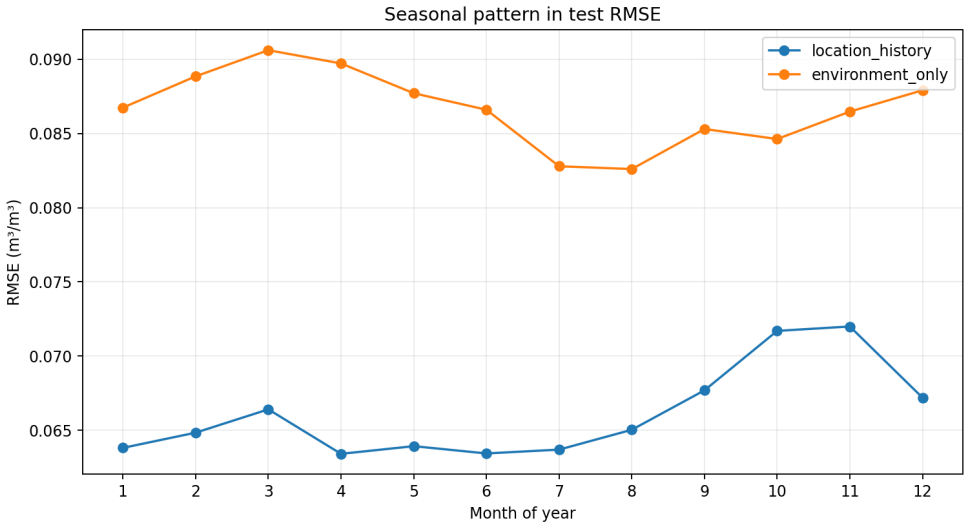


Figure 12.2: Month-of-year test RMSE. This view separates calendar seasonality from the irregular distribution of individual years.

Seasonal variation can arise from changing vegetation, cloud availability, snow or freeze conditions, rainfall-event frequency, acquisition geometry, and the depth response of the target. Interpreting a seasonal curve therefore requires the acquisition-age and missingness diagnostics alongside it.

12.3 Moisture range and residual structure

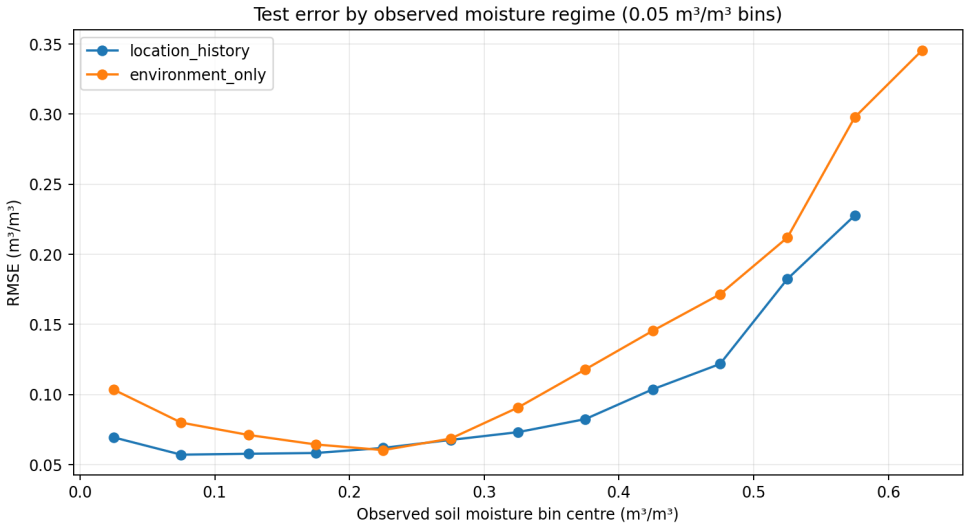


Figure 12.3: RMSE by target-moisture bin. A global score can hide whether the model performs differently in dry, intermediate, and wet states.

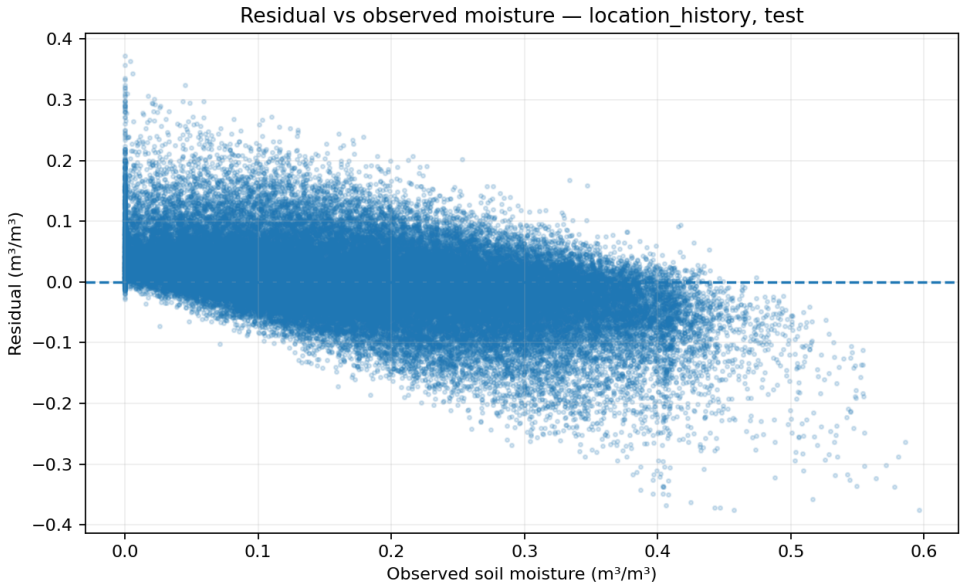


Figure 12.4: Residual versus target for location-history. Curvature or funnel shape would indicate remaining scale-dependent error.

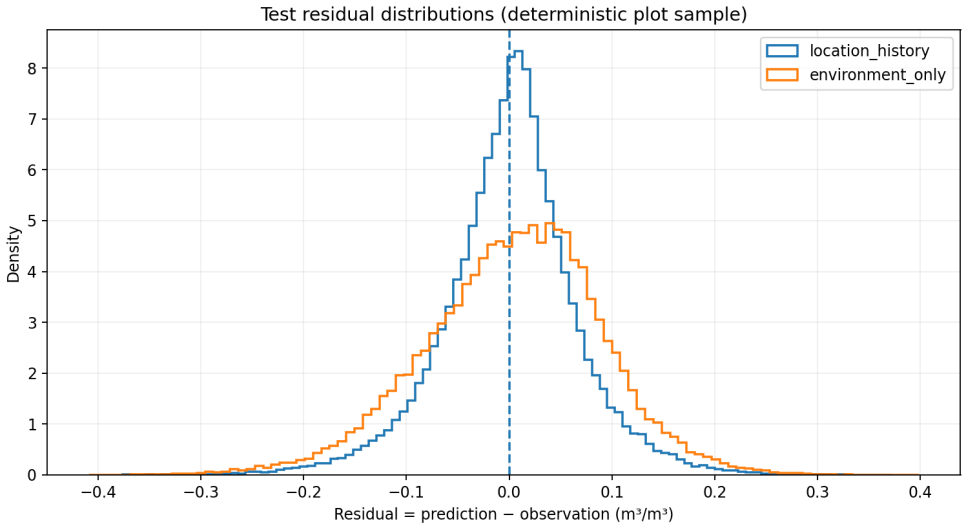


Figure 12.5: Location-history residual distribution. The centre, asymmetry, and tail are distinct pieces of evidence.

12.4 Raw versus calibrated output

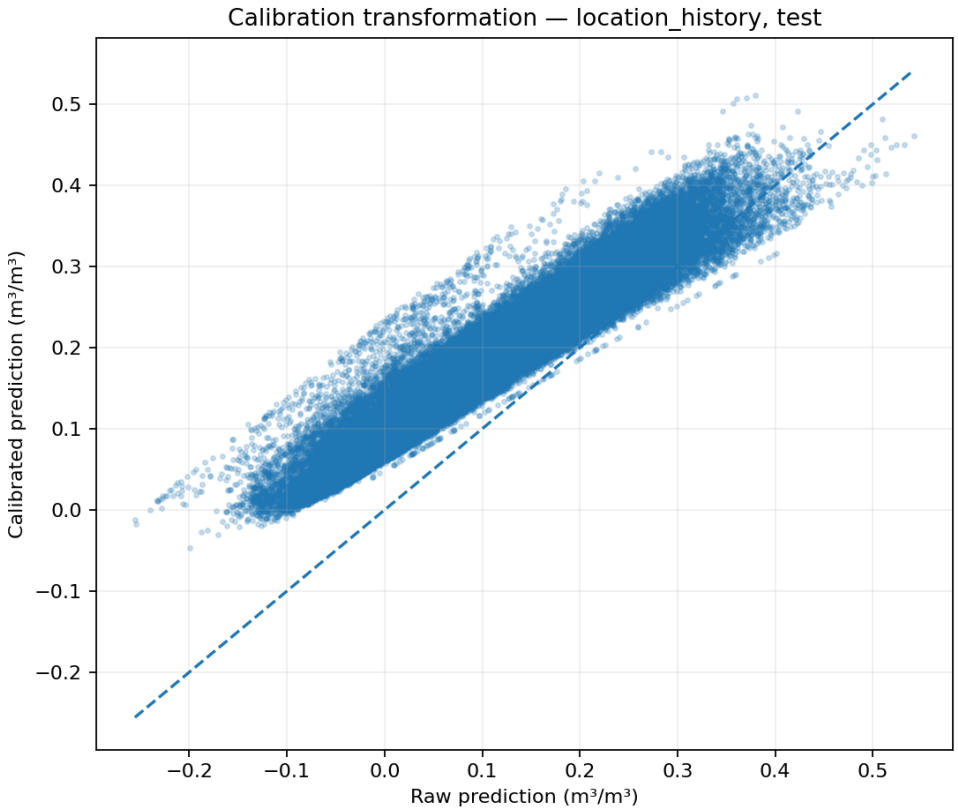


Figure 12.6: Raw and calibrated location-history predictions. Calibration is a measurable part of the production system.

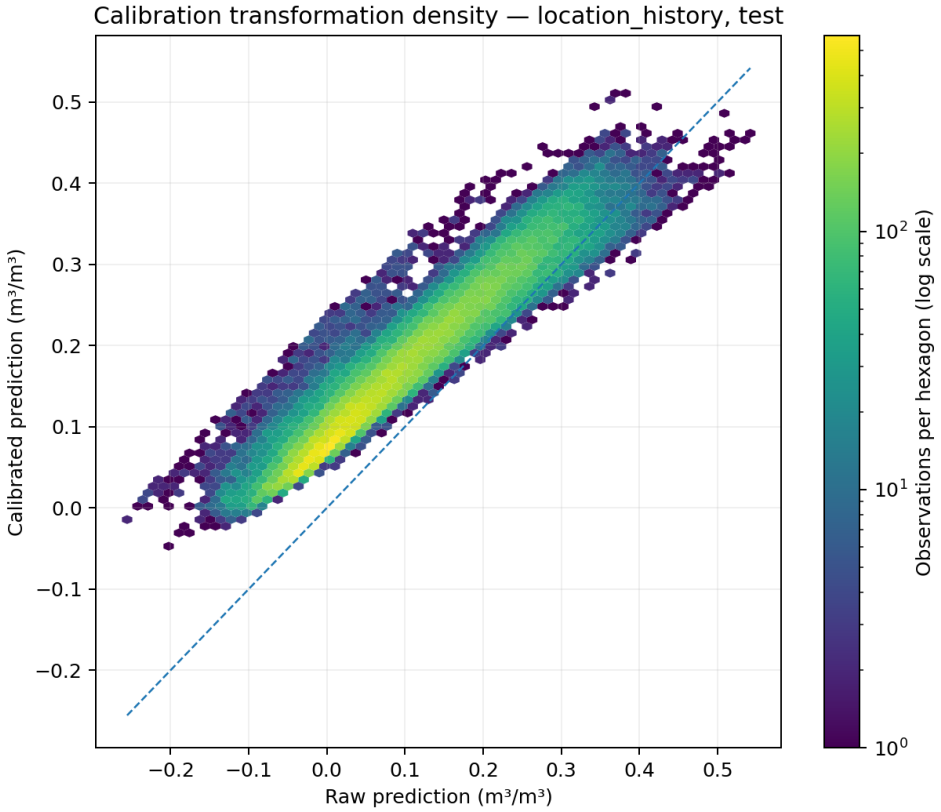


Figure 12.7: Density comparison before and after calibration. The visual shift illustrates why reporting only the final calibrated score is incomplete.

The raw-to-calibrated transformation is scientifically informative. A model can learn useful ranking and conditional structure while producing an output with incorrect gain or offset. The calibrator can correct this numerical mismatch, but its coefficients must be fitted on development data and frozen before an untouched certification test.

12.5 Paired station and regional comparisons

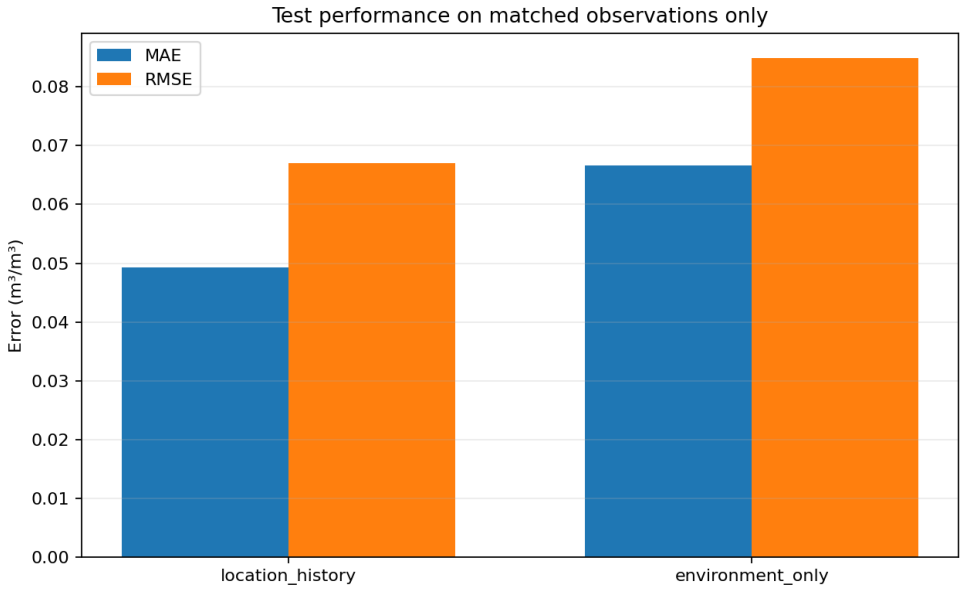


Figure 12.8: Paired configuration comparison. Pairing removes some confounding caused by different station populations.

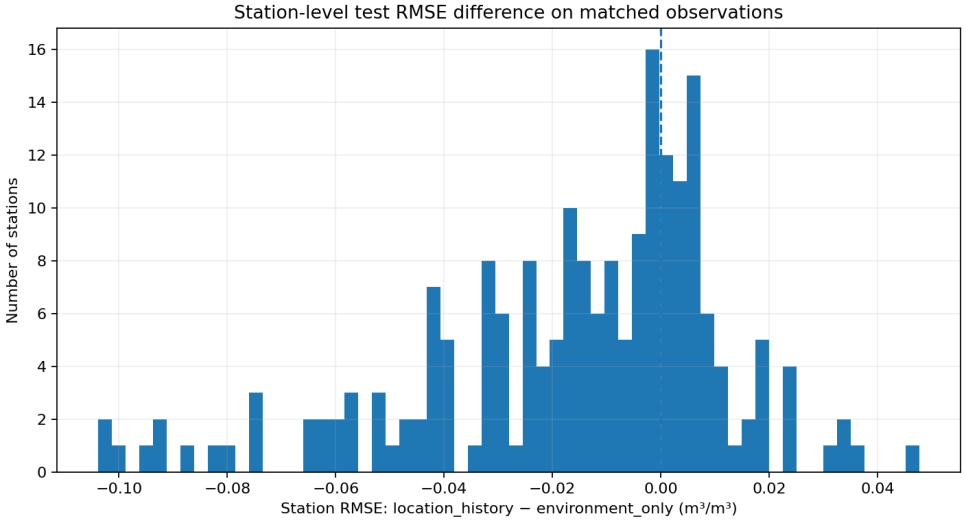


Figure 12.9: Station-level paired RMSE differences. Improvement is not spatially uniform, which is why the aggregate result needs station diagnostics.

12.6 Worst-station and event-context analyses

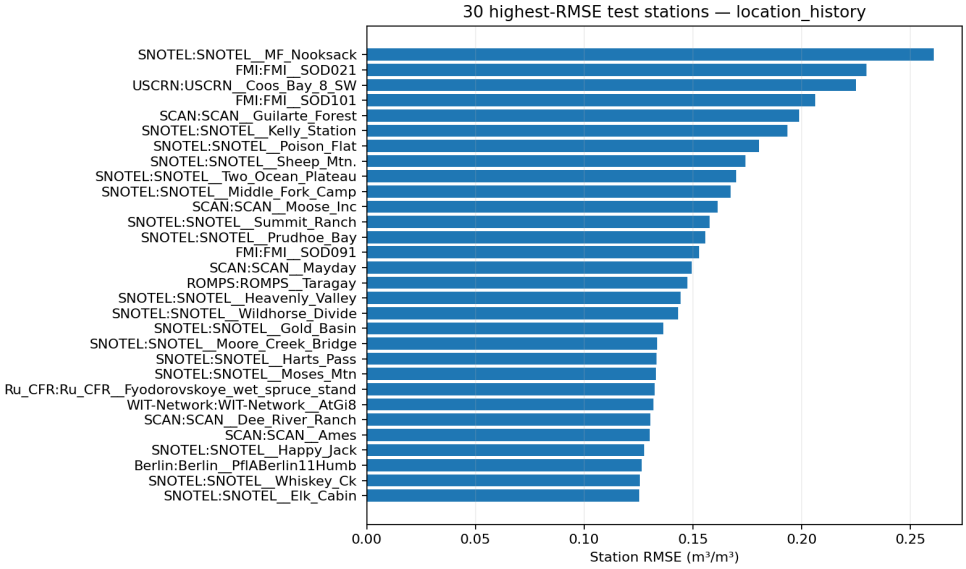


Figure 12.10: Worst 30 location-history stations. These are investigation targets, not evidence that the whole model fails in the same way.

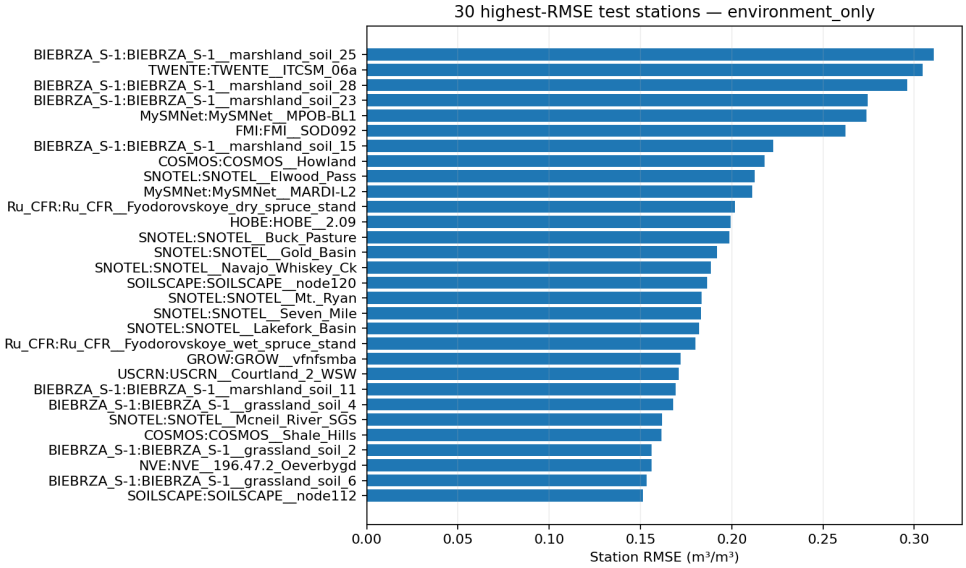


Figure 12.11: Worst 30 environment-only stations. Comparing the lists shows whether location/history changes the identity of difficult regimes.

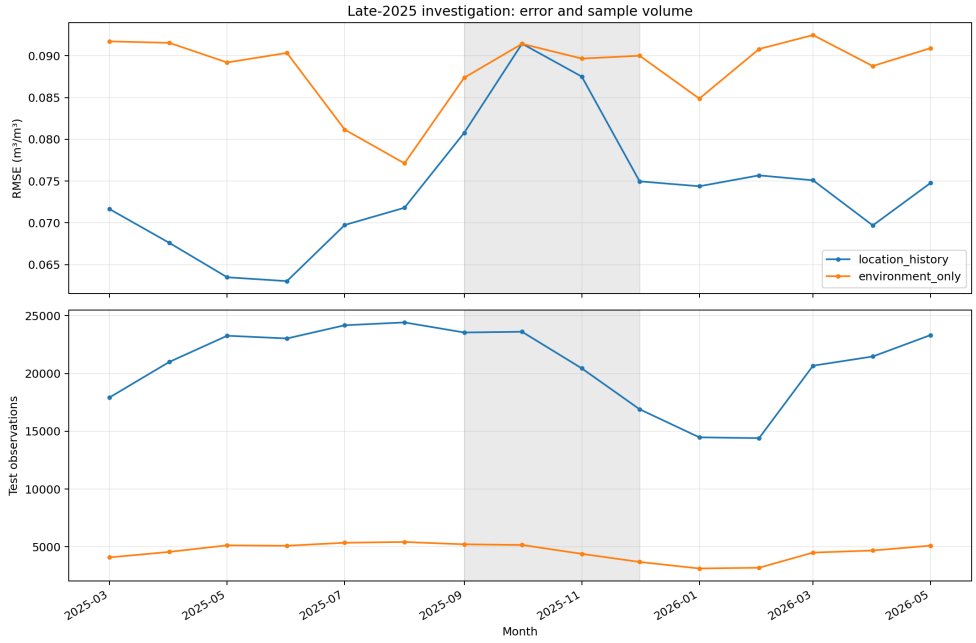


Figure 12.12: Late-2025 event context. Error spikes need event-level inspection before being attributed to a model architecture.

12.7 What the gallery establishes

The diagnostic gallery establishes that the result is not a single accidental number. It exposes temporal variation, station heterogeneity, range dependence, calibration dependence, and the long tail of hard cases. It does not by itself establish causal physical response or area-weighted global accuracy. Those claims require additional experimental design, especially block uncertainty and ecological holdouts.

Chapter 13

Technical Hurdles and Solutions

13.1 Data association was harder than tensor arithmetic

A tensor can have the correct rank, dtype, and finite values while belonging to the wrong station and date. The project therefore treated identity as a scientific product. Station/date keys, cell keys, row indexes, manifests, and sidecar tables survived across extraction, merging, caching, and training.

13.2 Same date is not necessarily same physical state

An ISMN daily mean and a nearest-valid Sentinel-1 acquisition can be separated by rainfall, evaporation, irrigation, freezing, or drainage. The resulting error contains temporal representativeness uncertainty. Strict same-day matching would reduce this uncertainty but also reduce global coverage.

13.3 Calibration can dominate the headline score

The raw location-history model is poorly scaled relative to the final target, while calibration makes it useful. The complete system therefore deserves the calibrated score, but the representation should not be credited with all of the calibration improvement. Raw and calibrated metrics are both reported.

13.4 Location is useful and dangerous

Coordinates and station history help represent broad climate and soil regimes. They can also encode site identity. A network-disjoint split limits direct network leakage, but does not prove transfer to unseen ecological or continental regimes. This is why the environment-only comparison is a critical control.

13.5 Missingness can become a hidden predictor

Validity masks preserve the difference between a low measurement and no measurement. However, the mask itself can correlate with season, cloud cover, region, sensor operation, or network. A model can exploit availability rather than physical signal. Missingness-stratified performance and mask-only baselines are appropriate follow-up tests.

13.6 Physical validity exceeds numerical validity

Finite-value and range checks do not prove hydrological plausibility. Strong aggregate R^2 can coexist with implausible wetting transitions, spatial discontinuities, or climate-specific failures. Event timing, persistence, moisture-bin residuals, texture-stratified errors, and monotonic stress tests are needed for stronger physical claims.

Chapter 14

Reproducibility, Audit Trail, and Release

14.1 What is recorded

The release surface includes:

- contract and feature schema;
- train/validation/test manifests;
- train-only scaler files and hashes;
- cache build signatures;
- Zarr and memory-map manifests;
- checkpoint configuration and SHA-256 hash;
- prediction Parquet outputs;
- metric tables and analysis manifest;
- exact-versus-overview strategy namespaces;
- Azure transfer checksums and completion metadata.

These records answer different reproducibility questions. The contract says what a value is supposed to mean. The split manifest says which identities were allowed to influence fitting, selection, and testing. The scaler hash says which numerical transformation was applied before the model saw a value. The cache signature says which producer, precision, fill convention, and source namespace created the array. The checkpoint hash says which

learned parameters generated the prediction. Reproducibility is therefore a chain of compatible identities rather than a single random seed.

The chain is also designed for diagnosis. If a metric changes, the audit trail allows the change to be narrowed to the source catalog, daily aggregation, remote asset selection, raster alignment, cache construction, model state, or metric script. Without this separation, a changed score would be visible but its cause would not be. That distinction is especially important in a multi-provider experiment where a source catalog or remote asset can change independently of the model code.

14.2 Failure should be explicit

The builders reject missing columns, duplicate station/date rows, unsupported split labels, missing features, wrong shapes, mixed temporal lengths, non-finite arrays, and invalid numeric targets. This is preferable to silently coercing bad input into a success-shaped training set.

The checks are placed at boundaries because errors become harder to localise after data have been merged. At the archive boundary, malformed files and unsupported depth metadata are rejected. At the daily-target boundary, duplicate identities and invalid quality flags are exposed. At the raster boundary, shape, row-order, source-asset, and valid-pixel checks are applied. At the tensor boundary, feature dimensions, masks, split IDs, and finite values are checked. At the evaluation boundary, prediction counts and key uniqueness are compared with the expected manifest.

This boundary design also distinguishes absence from failure. A missing remote observation can be a valid scientific condition and is represented by a mask. A missing required shard, a mismatched row count, or a stale cache signature is a pipeline failure and must stop the run. Treating both cases as a generic zero would make the model appear complete while changing the physical meaning of the input.

14.3 Reproduction sequence

1. Verify the contract and source manifests.
2. Rebuild or validate the catalog and daily Parquet.
3. Verify ERA5, SMAP, MODIS, Sentinel-1, and static-product provenance.
4. Validate cluster inventory, shards, checksums, and Zarr metadata.
5. Reconstruct train-only scalers and cache signatures.

6. Validate sample IDs, row order, shapes, and finite values.
7. Restore the exact checkpoint and model configuration.
8. Recompute predictions and compare full-count metric tables.

The sequence is intentionally ordered from meaning to computation. Rebuilding the neural network before confirming the target contract would reproduce a number without proving that it refers to the same task. Rebuilding the cache before checking the producer row order could reproduce a tensor shape while silently permuting the station associations. The first stages are therefore slower and more administrative than model loading, but they prevent a fast reproduction of the wrong experiment.

For a complete rerun, each stage should emit a compact manifest containing inputs, outputs, counts, hashes, elapsed time, and the reason for any skipped work. A skipped unit is reproducible only when its prior completion record is still valid under the current contract and source namespace. Otherwise it must be rebuilt. This is the operational meaning of resumability: reuse is allowed only when equivalence has been demonstrated.

14.4 Change control across providers

The data providers evolve on different schedules. A remote catalogue can expose a new asset while the station archive remains unchanged; a processing library can alter how a COG window is decoded; and a model notebook can change its default precision without changing its filename. The release process therefore records provider, collection, asset identifier, acquisition date, access strategy, and producer version separately. A filename alone is not considered an adequate provenance key.

When a source changes, the correct response depends on the layer affected. A new static soil layer requires rebuilding the static feature table but not necessarily the dynamic scene inventory. A new Sentinel asset can require re-running only the clusters whose search results changed. A scaler or fill policy change invalidates the normalised cache and all dependent tensors, even if the source arrays are byte-for-byte identical. This dependency structure prevents both unnecessary global rebuilds and unsafe partial reuse.

The same principle applies to software. A numerical library update is tested first on a bounded sample with known row counts, valid-pixel counts, shapes, and summary statistics. Only after those invariants match is the new producer allowed to write a production namespace. This is the data-engineering equivalent of checking a new numerical integrator against a solved test problem before using it in a long simulation.

14.5 Why auditability matters scientifically

Auditability is not only a software quality property. It determines whether a physics interpretation can be challenged and reproduced. If a location-history model improves over an environment-only model, the reader needs to know whether the improvement came from temporal memory, local static context, station identity, a calibration transformation, or a difference in available rows. The manifests and paired evaluation tables make those alternatives separable.

The same applies to a failure. A low score may reflect poor representation, temporal mismatch, missingness, a remote extraction gap, or a target aggregation problem. A reproducible pipeline does not guarantee that the model is correct, but it ensures that competing explanations can be tested against the same intermediate artifacts. That is the practical meaning of making a global machine-learning experiment scientifically inspectable.

Chapter 15

Conclusion

The global soil-moisture project is a substantial technical accomplishment because it integrates scientific data interpretation, cloud-scale acquisition, storage engineering, numerical optimisation, machine learning, and evaluation into one auditable system. The model score is only the visible endpoint. The deeper result is that a global experiment involving heterogeneous station observations, multi-resolution remote products, reanalysis, static layers, irregular histories, and finite compute was made operationally possible.

The engineering strategy is coherent:

1. define the physical target before extracting predictors;
2. preserve identity and provenance through every representation;
3. share remote work through clustering and reuse;
4. make interrupted work resumable and checksummed;
5. move deterministic preparation into persistent caches;
6. train a modality-aware model under bounded memory;
7. evaluate at full population scale while exposing limitations.

The final location-history configuration achieves calibrated RMSE 0.06594 and $R^2 = 0.65341$ over 905,248 test rows, compared with RMSE 0.08626 and $R^2 = 0.47046$ for the environment-only configuration. These results are meaningful because they are attached to a documented data contract, explicit provenance, extensive error analysis, and a clear statement of what the scores do and do not establish.

When the unoptimised experiment is impossible, optimisation becomes part of the scientific methodology. Here, reducing redundancy without erasing provenance converted computational feasibility into repeatable evidence.

Chapter 16

Future Work

16.1 Certification of physical transfer

The strongest next step is a set of evaluation protocols that separate interpolation among familiar regimes from transfer to genuinely novel regimes. The existing network-disjoint split is valuable, but future experiments should also hold out continents, climate zones, biomes, soil-texture regimes, and station density classes. These tests would show whether the location-history branch is learning transferable environmental structure or primarily regional priors.

16.2 Depth-specific and temporally synchronous targets

Headline metrics should be separated for 0–5 cm and 5–10 cm observations. The predictor-target acquisition age should be recorded as a first-class feature, and errors should be stratified by the offset between the target day and Sentinel-1 or MODIS support window. A strict same-day sensitivity experiment would quantify the price of temporal synchrony in coverage.

16.3 Uncertainty and statistically dependent observations

Station time series are autocorrelated, so confidence intervals should be estimated by station or network blocks rather than by treating every row as independent. Conformal prediction, ensemble dispersion, or a calibrated heteroscedastic head could provide uncer-

tainty intervals. These intervals should be evaluated for coverage during missing-data, event, and acquisition-age regimes.

16.4 Physical stress tests

Future analysis should test wetting after precipitation, drying under high VPD, persistence across short gaps, response across soil texture, and behaviour near dry and wet bounds. These tests would not impose an unjustified deterministic law on noisy observations; they would identify clearly implausible response patterns that aggregate regression scores can conceal.

16.5 Representation and approximation ablations

The exact native Sentinel-1 path should be compared directly with the overview-derived path. FP32 and unclipped cache variants should be evaluated against the production FP16 representation. Modality ablations should remove EVI, LST, radar, temporal history, static context, location, masks, and the SMAP anchor one at a time. This would quantify which technical approximations and information pathways are responsible for the final improvement.

16.6 Locked certification release

The final certification workflow should fit all scalers, calibration coefficients, hyperparameters, and checkpoint choices using training and validation only, then evaluate once on a locked test cohort. The current report records the historical test-R^2 selection policy transparently; a future locked release would provide a cleaner estimate of untouched generalisation.

16.7 Operational data system

The resumable acquisition system can be extended with automatic provenance lineage, remote asset version snapshots, formal schema migrations, and hardware-specific throughput dashboards. The goal would be to make every production prediction traceable to a source asset, acquisition time, cache signature, model checkpoint, and calibration version.

References

Vishal Batchu, Grey Nearing, and Varun Gulshan. A deep learning data fusion model using sentinel-1/2, soilgrids, smap, and gldas for soil moisture retrieval. *Journal of Hydrometeorology*, 24(10):1789 – 1823, 2023. doi: 10.1175/JHM-D-22-0118.1. URL https://journals.ametsoc.org/view/journals/hydr/24/10/JHM-D-22-0118.1.xml.

Kuai Fang, Chaopeng Shen, Daniel Kifer, and Xiao Yang. Prolongation of smap to spatiotemporally seamless coverage of continental u.s. using a deep learning neural network. *Geophysical Research Letters*, 44(21):11,030–11,039, 2017. doi: https://doi.org/10.1002/2017GL075619. URL https://agupubs.onlinelibrary.wiley.com/doi/abs/10.1002/2017GL075619.

GCOS Austria. Austrian gcos report 2025. https://www.gcos.at/reports/Austrian_GCOS_Report_2025.pdf, 2025. Inventory report for the Austrian contribution to the Global Climate Observing System.

Dharmendra Pandey, Prashant Srivastava, Rucha Dave, Praveen Gupta, and Rashmi Sharma. Improvised sar time series algorithm for nisar high resolution soil moisture product over indian tropical region. pages 2111–2115, 08 2025. doi: 10.1109/IGARSS55030.2025.11313937.

Zeyang Wei, Lifei Wei, Ting Wang, Qikai Lu, Shuang Tian, Fei Zhang, and Yanfei Zhong. A global long-term daily multilayer soil moisture dataset derived from machine learning. *Scientific Data*, 13(1):122, Dec 2025. ISSN 2052-4463. doi: 10.1038/s41597-025-06436-0. URL https://doi.org/10.1038/s41597-025-06436-0.

Wei Xu, Zhaoxu Zhang, Zehao Long, and Qiming Qin. Downscaling smap soil moisture products with convolutional neural network. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 14:4051–4062, 2021. doi: 10.1109/JSTARS.2021.3069774.